

HYPERGRID

N-Dimensional Distributed Spreadsheet System

Platform-Agnostic Software Design Document

Version 1.0 · March 2026

*Rows · Columns · N Custom Dimensions · N-Attribute Cells · Graphs & Networks
Spaces · Workspaces · Namespaces · Multi-Identity · CRDT · Link Forest · AI Intelligence*

Attribute	Value
System Name	Hypergrid — N-Dimensional Distributed Spreadsheet System
Acronym	NDSS / Hypergrid
Version	1.0
Status	Active Design — Reference Architecture
Audience	Platform architects, system engineers, product teams adopting Hypergrid
Primary Languages	Rust (core substrate) · Go (services) · TypeScript (SDK/UI) · SQL (PostgreSQL)
Core Modules	HG-CORE · HG-DIM · HG-CELL · HG-VIEW · HG-COMP · HG-CRDT · HG-GRAPH · HG-SPACE · HG-ID · HG-NS · HG-EXPORT · HG-AI
License	Apache 2.0 (reference implementation) · Commercial license available

Table of Contents

1. Overview & Design Philosophy

1.1 What is Hypergrid?

Hypergrid is a platform-agnostic, open-architecture, N-dimensional distributed spreadsheet system. It provides any application platform — productivity tools, project management systems, business intelligence platforms, operating systems for organizations, marketplaces, social networks, or domain-specific SaaS products — with a universal, infinitely extensible data substrate that models all entities, relationships, computations, and intelligence as a single, coherent, living grid.

A conventional spreadsheet has two dimensions: rows and columns. Hypergrid generalizes this to N dimensions: rows are Dimension 1, columns are Dimension 2, and any number of additional custom dimensions — time slices, geographic regions, organizational units, product versions, personas, risk categories, or any domain-specific axis — can be declared as Dimension 3 through N. Every cell in this N-dimensional grid is itself an N-attribute data point, carrying not just a single value but a structured map of typed attribute keys.

The central insight of Hypergrid: a spreadsheet is a universal model. Anything that can be described as "an entity with properties (columns), organized in groups (rows), across contexts (dimensions)" is a natural fit for a spreadsheet substrate. Hypergrid makes that substrate distributed, collaborative, intelligent, connected, and infinitely extensible — for any platform that needs it.

1.2 Core Abstractions

Abstraction	Traditional Spreadsheet	Hypergrid Generalization
Workbook	A single file containing multiple sheets	A Grid — the root container for all sheets/cubes within a tenant or namespace
Sheet	A 2D grid of rows × columns	A Hypercube — an N-dimensional grid: Dim1 × Dim2 × ... × DimN. Every Hypercube is a view/projection of the Universal Cell Store.
Row	A horizontal sequence of cells, first dimension	Dimension-1 Axis — a typed, indexed, governed slice of the Hypercube along the primary entity axis
Column	A vertical sequence of cells, second dimension	Dimension-2 Axis — a typed, indexed, governed slice of the Hypercube along the property/attribute axis
Extra dims	(none)	Dimension-3...N — custom, named, typed, indexed axes: Time Geography OrgUnit Version Persona RiskTier Custom
Cell	A single value at the intersection of one row and one column	A HyperCell — a data point at the intersection of N dimension keys, carrying N typed attribute keys (value, type, formula, version, computed_by, visibility, tags, and any domain-defined key)

Abstraction	Traditional Spreadsheet	Hypergrid Generalization
Formula	A function over other cells in the same sheet	A ComputedAttribute — a function over any cell attributes across any dimension combination, including AI-derived signals and cross-grid joins
Filter	A criterion applied to rows	A DimSlice — a predicate applied to any combination of dimension axes, producing a sub-cube or flattened projection
Sheet tab	A named view over the same 2D grid	A HypercubeView — a saved projection, filter, sort, group, and rendering configuration over any N-dimensional sub-space
Workbook link	An external reference to another workbook	A GraphEdge — a typed, directed edge in the Hypergraph connecting cells, rows, or full grids across tenants, namespaces, or platforms

1.3 Module Inventory

Module	Code	Description
Core Substrate	HG-CORE	Universal data model: Grid, Hypercube, DimensionAxis, HyperCell, HyperRow, EventLog, CrdtLog, VectorClock, PolicyEngine
Dimension System	HG-DIM	Declare, type, index, and govern custom dimensions. DimensionAxis registry. Sparse/dense encoding. Cross-dim join planning.
Cell Attribute Model	HG-CELL	N-attribute cell: attribute key registry, typed attribute values, attribute-level CRDT, computed attributes, inheritance.
View Engine	HG-VIEW	HypercubeView: N-dim filter (DimSlice), sort, group, pivot, projection, DimFold, board modes, rendering adapters.
Computation Engine	HG-COMP	ComputedAttribute evaluation: synchronous arithmetic tier and asynchronous AI/ML tier. Formula language. Rollup engine.
Distributed Consistency	HG-CRDT	N-dim CRDT: LWW per attribute, OR-Set for set-valued attributes, custom lattice for status axes, VectorClock per federation node.
Graph & Network	HG-GRAP H	Hypergraph: typed edges between HyperCells/HyperRows/Grids. Link forest and tree traversal. Shadow cell protocol. Mirror attribute sync.
Spaces & Workspaces	HG-SPAC E	Bounded operational contexts (Spaces) and active working sessions (Workspaces). Governance. Treasury. Space-link sub-graph.
Identity & Multi-Tenancy	HG-ID	Multi-tenant, multi-identity, multi-profile model. TenantPartition, VisibilityMask, SplitPolicy, CrossTenantMerge.
Namespace System	HG-NS	Hierarchical addressing for all entities. NamespacePath URI scheme. Resolution protocol. Federation sync.
Export & Integration	HG-EXPO RT	CSV, XLSX, JSON, JSON-LD, Parquet, Arrow, GraphQL, webhook, embed, streaming export of any N-dim view.
AI Intelligence Layer	HG-AI	Pluggable intelligence engines: anomaly detection, health scoring, recommendation, prediction, NL-to-query translation.

Module	Code	Description
Query Language	HG-QL	HyperQL — N-dimensional query language: SELECT over any dim combination, DimSlice predicates, AS_OF time travel, JOIN, SHADOW INCLUDE.
Plugin System	HG-PLUGIN	HypercubePlugin trait for custom dimension types, attribute types, computed models, rendering adapters, and data connectors.
Telemetry & Ops	HG-OPS	Distributed tracing, metrics, structured audit logs, health probes, federation sync health.

2. N-Dimensional Data Model

The N-dimensional model is the theoretical foundation of Hypergrid. Every concept — rows, columns, cells, sheets, formulas, filters, groups — is a special case of the general N-dimensional framework. Conventional 2D spreadsheet operations remain available as the most common projection of this framework.

2.1 Formal Definition

Let a Hypercube H be defined by an ordered tuple of N dimension axes:

```
H = (D1, D2, ..., DN)

where:
  Di = DimensionAxis(id, name, type, key_set, ordering, cardinality)
  N   = the dimensionality of the Hypercube (N ≥ 2)

A HyperCell C is a data point addressed by a coordinate tuple:
  C = Cell(k1 ∈ D1.key_set, k2 ∈ D2.key_set, ..., kN ∈ DN.key_set)

Each HyperCell carries an attribute map:
  C.attributes = { a1: v1, a2: v2, ..., aN: vN }
  where ai ∈ AttributeKeyRegistry and vi is a typed value

The Universal Cell Store UCS holds all HyperCells across all Hypercubes:
  UCS = { (grid_id, dim_keys_tuple) → HyperCell }

A conventional 2D spreadsheet is the N=2 special case:
  D1 = RowAxis      (entity dimension)
  D2 = ColumnAxis   (attribute/property dimension)
  C.attributes = { "value": V, "type": T, "format": F } -- conventional cell
```

2.2 Dimension Axis Model

Every axis of a Hypercube is a typed, indexed, governed DimensionAxis. The axis defines the key space (what values can appear in that dimension), the ordering of keys, and the cardinality (dense vs. sparse).

Field	Type	Description
axis_id	UUID	Globally unique identifier for this axis within the grid

Field	Type	Description
name	String	Human-readable name: "rows", "columns", "time", "region", "org_unit", etc.
dim_index	u8 (1–255)	Position in the ordered dimension tuple. D ₁ =1 (rows), D ₂ =2 (columns), D ₃ –D _∞ =custom.
axis_type	AxisType enum	EntityAxis PropertyAxis TimeAxis GeoAxis CategoryAxis OrdinalAxis HierarchyAxis GraphAxis Custom(String)
key_type	KeyType enum	Uuid String Integer Ordinal Timestamp GeoHash HierarchyPath Custom — the type of keys in this dimension
key_set	KeySet	The set of valid keys for this axis. Enumerated (dense) or schema-constrained (sparse/infinite).
key_cardinality	KeyCardinality	Dense(max_keys) Sparse Infinite — affects storage encoding and index choice
ordering	AxisOrdering	Unordered Lexicographic Numeric Chronological Custom(CompareFn) — determines sort behavior across this axis
hierarchy	Option<HierarchyDef>	If AxisType=HierarchyAxis: parent-child relationships between keys. Enables group-by and drill-down across this axis.
visibility	AxisVisibility	Public Tenant Private — governs which tenants/identities can see this axis in shared grids
nullable	bool	If true: HyperCells need not exist for every combination including this axis (sparse). If false: every key must have a cell (dense).
default_attribute	Option<AttributeKey>	When this axis is rendered as a 2D spreadsheet, which cell attribute is shown as the primary "value"
crdt_semantics	CrdtSemantics	LWW OR-Set Counter Lattice — CRDT behavior for mutations along this axis
index_strategy	IndexStrategy	BTree Hash GIN BRIN TimeSeries Spatial — storage index type for this axis
plugin_id	Option<PluginId>	If this is a custom axis type: the HypercubePlugin that provides key validation, rendering, and computation
created_at	DateTime<Utc>	Axis registration timestamp
metadata	Map<String, Value>	Extensible axis-level metadata for platform-specific configuration

2.3 Built-in Axis Types & Their Properties

AxisType	Key Type	Ordering	Typical Use	Special Behaviors
EntityAxis	Uuid / String	Unordered / Custom	D ₁ (rows) — primary entity dimension: users, projects,	Full PortfolioRow/record model.

AxisType	Key Type	Ordering	Typical Use	Special Behaviors
			products, events, transactions	GraphEdge::Hierarchy supported between keys. CRDT: LWW per attribute.
PropertyAxis	String	Lexicographic/Custom	D_2 (columns) — property/attribute dimension: field names, metric names, column labels	Column type registry (see §4). ComputedAttribute support. VisibilityMask per column key.
TimeAxis	Timestamp	Chronological	D_3 — temporal slicing: snapshots, time-series, event history, fiscal periods	AS_OF time-travel queries. Automatic snapshot generation. BRIN index. Rollup by hour/day/week/month/year.
GeoAxis	GeoHash / Coord	Spatial	D_4 — geographic slicing: region, country, city, custom polygon, lat/lng bucket	Spatial index (PostGIS/GeoHash). Drill-down: continent → country → region → city. Spatial join support.
CategoryAxis	String (enum)	Ordinal / Custom	D_3 -N — classification: product line, risk tier, department, scenario, cohort	Enum key validation. Hierarchy support. Cross-category pivot.
OrdinalAxis	Integer	Numeric	D_3 -N — ranked/ordered slicing: priority, ranking, version number, sequence	Dense numeric key space. Efficient range queries. Used for versioned cubes.
HierarchyAxis	HierarchyPath	Tree (depth-first)	D_3 -N — org chart, product taxonomy, file system, tag hierarchy, geographic admin levels	Parent-child key relationships. Rollup propagation. Drill-down/drill-up operations.
GraphAxis	Nodeld	Graph-order / Custom	D_3 -N — graph traversal dimension: network hops, dependency layers, influence tiers	KLNK/Hypergraph integration. Keys are graph node IDs. Enables adjacency matrix representations.
VersionAxis	SemVer / Integer	Numeric	D_3 — schema/data versioning: parallel versions of the same cube across schema migrations	Fork-and-merge version operations. Diff between version keys. Schema migration tracking.
TenantAxis	TenantId	Unordered	D_3 — multi-tenancy: share a single Hypercube across tenants while enforcing RLS per key	Row-level security: each key visible only to matching tenant. Used for SaaS shared-cube deployments.
PersonaAxis	String	Custom	D_3 -N — audience/persona slicing: user segments, role types, maturity levels, view contexts	A/B testing support. Personalized ComputedAttributes. Feed personalization.
ScenarioAxis	String	Custom	D_3 -N — what-if and planning: base, optimistic,	Scenario comparison views. Variance computed

AxisType	Key Type	Ordering	Typical Use	Special Behaviors
			pessimistic, budget, forecast, actuals	across scenario keys. Plan-vs-actual analysis.
Custom	Defined by plugin	Plugin-defined	Any domain-specific axis not covered above: genome position, musical note, physics parameter	HypercubePlugin provides key_type, validation, index, rendering, and computation handlers

2.4 Dimensionality Constraints

Constraint	Value	Rationale
Minimum dimensions (N)	2 (D_1 =EntityAxis, D_2 =PropertyAxis)	2D is the degenerate case: a conventional spreadsheet. All Hypergrid operations are valid at N=2.
Maximum dimensions (N)	16 (hard limit)	Beyond 16 dims, query planning complexity exceeds practical utility. Recommended max: 6 for most use cases.
Max keys per dense axis	10,000,000 (10M)	Dense axes are fully materialized. Beyond 10M keys, use SparseAxis with on-demand materialization.
Max keys per sparse axis	Unlimited (constrained by storage)	Sparse axes store only cells with non-null attributes. Storage: $O(\text{non-null cells})$.
Max attributes per HyperCell	1,024	Attribute key registry limit per grid. In practice: 20–150 attributes per cell are typical.
Max concurrent editors per cube	1,000	CRDT buffer capacity. Beyond 1K concurrent writers: federation node sharding recommended.
Max federation peers per grid	255	VectorClock size. Each peer is one slot in the VectorClock map.
Max custom dimensions added at runtime	Unlimited (up to N=16 total)	Custom axes can be added to any Hypercube post-creation. Existing cells gain the new axis with null/default attribute values.

2.5 Dimensional Sparsity & Encoding

Hypergrid supports two storage encodings for the cell store, chosen at axis declaration time based on the axis's expected density:

Encoding	When To Use	Storage Schema	Query Behavior
Dense Encoding	Axis is small and fully populated. Every ($D_1 \times D_2$) row-column pair is expected to have a value. $N=2$ conventional spreadsheets always use dense encoding for $D_1 \times D_2$.	N-dim array storage: PostgreSQL ARRAY or columnar layout. $O(D_1.\text{card} \times D_2.\text{card} \times \dots)$ cells allocated.	$O(1)$ cell lookup by coordinate. Range scans are contiguous. Aggregations are SIMD-friendly.
Sparse Encoding	Axis is large or partially populated. Most coordinate combinations have no meaningful value (null). Any axis with cardinality $> 100,000$ keys or TimeAxis/GeoAxis/GraphAxis.	Key-value storage: $\{(k_1, k_2, \dots, k_N): \text{attribute_map}\}$ in a hash-indexed PostgreSQL table. Only non-null cells stored.	$O(\log n)$ cell lookup by coordinate. Range scans require index traversal. Best for time-series and geo.
Hybrid Encoding	$D_1 \times D_2$ are dense; D_3-D_N are sparse. The most common production pattern: rows and columns are dense, custom time/geo/category axes are sparse.	Dense 2D layout for $D_1 \times D_2$ slice; sparse key-value index for D_3-D_N extension cells.	Fast conventional 2D access + efficient N-dim extension queries. Recommended for most use cases.

3. HyperCell & N-Attribute Model (HG-CELL)

A HyperCell is not a single value. It is an N-attribute data point — a structured map of typed attribute keys located at a specific coordinate in the N-dimensional grid. Conventional spreadsheet cells have 3 implicit attributes: value, type, and format. HyperCells generalize this to N attributes, each with its own type, visibility, computation, versioning, and CRDT semantics.

3.1 HyperCell Data Model

```
pub struct HyperCell {
    // Coordinate — uniquely identifies this cell in the N-dim grid
    pub grid_id:      GridId,          // parent Grid
    pub cube_id:      CubeId,          // parent Hypercube
    pub coord:        DimCoordinate,   // Vec<DimKey> — one key per dimension
    // coord example for N=4: (entity_id, column_name, "2026-Q1", "EMEA")

    // Attributes — the N-attribute data payload
    pub attributes:    AttributeMap,    // HashMap<AttributeKey, TypedAttrValue>

    // Versioning
    pub vector_clock:  VectorClock,     // causal ordering across federation nodes
    pub version:       u64,             // monotonic mutation counter
    pub created_at:    DateTime<Utc>,
    pub updated_at:    DateTime<Utc>,
    pub last_actor:    ActorId,         // "{tenant_id}:{identity_tag}:{node_id}"

    // Governance
    pub visibility:    CellVisibility,   // Public|Tenant|Identity|Private
    pub policy_ids:    Vec<PolicyId>,
    pub audit_trail:   EventLogRef,     // pointer to EventLog entry sequence

    // Graph integration
    pub edge_refs:     Vec<EdgeRef>,    // edges originating from this cell

    // Computed attribute cache
    pub computed_cache: HashMap<AttributeKey, CachedComputedAttr>,
}

pub struct DimCoordinate {
    pub keys: Vec<DimKey>, // ordered: [D1_key, D2_key, ..., DN_key]
    pub n:    u8,          // dimensionality — must match parent Hypercube.N
}

pub struct AttributeMap {
    pub entries: HashMap<AttributeKey, TypedAttrValue>,
}
```

```
// AttributeKey: String – registered in the cube's AttributeKeyRegistry
// TypedAttrValue: Text | Number | Currency | Bool | Date | Enum | Relation | Tag | Json | Null
}
```

3.2 Attribute Key Registry

Every attribute key used in any HyperCell must be registered in the Hypercube's AttributeKeyRegistry. The registry defines the type, default value, computation, visibility, and CRDT semantics for each attribute. This is the equivalent of a "column schema" generalized to apply across all dimensions.

Field	Type	Description
key	String	The attribute key name. Unique within the Hypercube. E.g. "value", "health_score", "owner", "tags", "budget", "status"
display_name	String	Human-readable label shown in UI headers and export column names
attr_type	AttributeType enum	Text Number Currency Percent Bool Date DateTime Duration Enum MultiEnum Relation MultiRelation User Tag Json Computed AI Formula Audit Custom
description	String	Documentation string for this attribute key
dimension_scope	DimScope	AllDimensions SpecificAxes(Vec<AxisId>) — which dimension axes this attribute applies to. Allows axis-specific attributes.
default_value	Option<TypedAttrValue>	Default value when this attribute is not explicitly set on a HyperCell
required	bool	If true: cells in the in-scope dimension must have this attribute set
visibility	AttrVisibility	Public Tenant Identity Owner — which observers can read this attribute
write_permission	PermissionTier	Minimum permission tier required to write this attribute. "Computed" and "AI" attrs are system-write-only.
crdt_semantics	CrdtSemantics	LWW OR-Set Counter Max Min Lattice(order_def) — conflict resolution strategy
computation	Option<AttrComputation>	If attr_type=Computed: synchronous formula expression. If attr_type=AI: engine plugin reference + model name.
aggregation	Option<Vec<AggFn>>	Supported aggregation functions: SUM AVG MIN MAX COUNT MEDIAN STDDEV MODE DISTRIBUTION
index_strategy	IndexStrategy	None BTree Hash GIN — storage index for this attribute key
inherits_from	Option<AttributeKey>	Attribute inheritance: if not set on a HyperCell, inherit from parent in hierarchy axis (HierarchyAxis types only)
version_track	bool	If true: maintain full history of this attribute's values in EventLog. If false: only current value stored.

Field	Type	Description
searchable	bool	If true: indexed by the full-text search engine
export_label	Option<String>	Override label used in CSV/XLSX/JSON export headers for this attribute
plugin_id	Option<PluginId>	For Custom attribute types: the plugin providing validation, rendering, and computation

3.3 Core Attribute Key Set (Built-in)

Every Hypercube includes the following built-in attribute keys on every HyperCell unless explicitly disabled:

Attribute Key	Type	Description	CRDT	Visibility
value	Any (configured per D ₂ column key)	The primary data value — the "cell value" in conventional spreadsheet terms. Type and computation determined by D ₂ column key registration.	LWW	Inherits cell visibility
data_type	Enum (AttributeType)	The runtime type of value. Allows heterogeneous cells in sparse grids.	LWW	Public
format	String (format pattern)	Display format string: "0.00%", "\$#,###.00", "YYYY-MM-DD", "#,###", etc.	LWW	Public
formula	Option<String>	Formula expression if this cell is formula-driven (FormulaAttribute). Null for data cells.	LWW	Tenant
computed_by	Option<String>	If type=AI Computed: the plugin/model that computed this value.	System	Tenant
computed_at	Option<DateTime>	Timestamp of last computation.	System	Tenant
confidence	Option<f32>	AI-computed confidence score (0.0–1.0) for computed attributes.	System	Tenant
version	u64	Monotonic mutation counter for this specific attribute on this cell.	Counter (max)	Public
visibility	CellVisibility	Per-cell visibility override. Default: inherits from row then cube default.	LWW	Owner
locked	bool	If true: no mutations permitted by non-owner actors. Used for audit-locked records.	LWW (Owner only)	Public
tags	Vec<String>	User-defined discovery tags on this specific cell (rare; more common on row-level cells).	OR-Set	Inherits

Attribute Key	Type	Description	CRDT	Visibility
comment_count	u32	Count of active comments on this cell (annotation system integration).	Counter	Tenant
change_highlight	Option<ChangeType>	UI signal: Added Modified Deleted Imported — shown as color indicator in current session.	Session-local	Session
error	Option<CellError>	Formula or computation error state: DivisionByZero CircularRef RefError ComputeError etc.	LWW	Tenant
anomaly_flag	Option<AnomalySignal>	AI-detected anomaly on this cell: OutlierValue SuddenChange PatternBreak etc.	AI-write	Tenant
last_modified_by	ActorId	Actor who last modified this cell's value attribute.	LWW	Tenant
modified_at	DateTime	Timestamp of last value modification.	LWW	Public
source	Option<String>	Data provenance: "manual", "import:csv", "api:service-name", "computed:model-name", "sync:peer-node"	LWW	Tenant
dim_context	Map<AxisId, ContextMeta>	Per-dimension context metadata for this cell. Allows cells to carry axis-specific annotations beyond the coordinate.	LWW per axis	Tenant

3.4 Domain Attribute Extensions

Platforms adopting Hypergrid define domain-specific attribute key extensions for their specific use cases. These are registered in the AttributeKeyRegistry and follow all the same CRDT, visibility, and computation rules as built-in attributes. Example domain extension sets:

Domain	Extension Set Name	Example Attribute Keys Added
Project Management	PM Extension	status, priority, assignee, due_date, story_points, sprint_ref, acceptance_criteria, cycle_time, lead_time, blocked_by
Financial / ERP	Finance Extension	budget_allocated, budget_spent, currency, cost_center, gl_account, tax_category, approval_status, payment_terms, invoice_ref
CRM	CRM Extension	lead_score, pipeline_stage, close_probability, deal_value, company_ref, contact_ref, last_contacted_at, next_action_due
HR / People Ops	People Extension	employment_type, department, start_date, end_date, compensation, performance_rating, skills, reporting_to, benefit_tier

Domain	Extension Set Name	Example Attribute Keys Added
Product / Engineering	Eng Extension	ticket_type, component, severity, environment, repro_steps, fix_version, test_coverage, code_owner, sla_tier
Scientific Research	Research Extension	experiment_id, hypothesis, measurement_unit, uncertainty, confidence_interval, replications, p_value, citation_refs
Healthcare	Clinical Extension	patient_id, diagnosis_code, procedure_code, provider_id, encounter_date, dosage, clinical_status, privacy_tier
Supply Chain	SCM Extension	sku, supplier_id, lead_time_days, safety_stock, reorder_point, lot_number, expiry_date, warehouse_location
Marketing	Marketing Extension	campaign_id, channel, impressions, clicks, conversions, cpa, roas, audience_segment, creative_ref, a_b_variant
Compliance / Legal	Legal Extension	regulation_ref, obligation_type, control_owner, last_audit_date, risk_rating, remediation_due, evidence_refs

4. Grid & Hypercube Architecture (HG-CORE)

4.1 Grid — The Root Container

A Grid is the top-level container for all Hypercubes, dimensions, attribute registries, tenants, namespaces, spaces, workspaces, and graph structures within a logical deployment unit. It is equivalent to a database in the RDBMS analogy, a workbook in the 2D spreadsheet analogy, or a portfolio system in a specific platform implementation.

```
pub struct Grid {
  pub grid_id:      GridId,           // UUID - globally unique
  pub name:         String,
  pub description:  Option<String>,
  pub tenant_id:    TenantId,         // top-level tenancy boundary
  pub namespace_root: NamespacePath, // root of this grid's NS tree
  pub cubes:        Vec<CubeId>,      // registered Hypercubes
  pub dim_registry: DimensionRegistry, // all axes defined for this grid
  pub attr_registry: AttributeKeyRegistry, // all attribute keys for this
grid
  pub graph:        HypergraphRef,    // this grid's node/edge store
  pub spaces:        Vec<SpaceId>,    // Spaces within this grid
  pub federation_peers: Vec<FederationPeer>, // CRDT peers for this grid
  pub policy_engine: PolicyEngineRef,
  pub ai_engine:     Option<AIEngineRef>, // pluggable AI/ML engine
  pub storage_config: StorageConfig,
  pub created_at:    DateTime<Utc>,
  pub metadata:      Map<String, Value>, // platform-defined extensions
}
```

4.2 Hypercube — The N-Dimensional Sheet

```
pub struct Hypercube {
  pub cube_id:      CubeId,
  pub grid_id:      GridId,
  pub name:         String,           // display name (e.g. "Projects")
  pub namespace_path: NamespacePath,  // NS address:
grid://tenant/cube/name/
  pub dimensions:   Vec<DimensionAxis>, // ordered D1...Dn
  pub n:            u8,                // dimensionality (2-16)
  pub schema_version: u32,             // incremented on dim/attr add
  pub encoding:      CubeEncoding,     // Dense | Sparse | Hybrid
  pub cell_count:    u64,              // non-null cells
  pub default_view_id: Option<ViewId>,
  pub views:         Vec<ViewId>,
  pub attr_overrides: AttrKeyOverrides, // cube-level attribute config
overrides
}
```



```

    pub governance:      CubeGovernance,      // access, approval, audit
settings
    pub space_id:        Option<SpaceId>,      // parent Space (if Space-scoped)
    pub status:          CubeStatus,          // Draft|Active|Archived|Migrating
    pub created_at:      DateTime<Utc>,
    pub updated_at:      DateTime<Utc>,
    pub metadata:        Map<String, Value>,
}

```

4.3 Dimension Addition & Schema Migration

One of Hypergrid's most powerful features is the ability to add new dimensions to an existing Hypercube at runtime without downtime or data migration. When a new axis D_{i+1} is added to a Hypercube with existing cells:

1. **Schema Migration:** A SchemaVersion bump is recorded. A SchemaEvent is appended to the EventLog.
2. **Default Key Assignment:** All existing HyperCells receive a default key for the new axis. The default is configured in the DimensionAxis declaration (e.g. "unclassified" for a CategoryAxis, "2026-Q1" for a TimeAxis).
3. **Sparse Handling:** If the new axis uses Sparse encoding, no cells are physically created. Existing cells simply have no explicit entry for the new axis — queries that include the new axis treat these cells as having the default key.
4. **Index Provisioning:** The appropriate index is provisioned asynchronously (background job). The cube remains fully queryable during provisioning via a pre-existing full-scan fallback.
5. **View Updates:** All existing ViewDefinitions are updated to include the new axis as a collapsed (default-value) DimSlice. Existing views appear unchanged to users until they explicitly expand the new axis.
6. **CRDT Broadcast:** A DimensionAdded CrdtOperation is broadcast to all federation peers. Peers apply the same migration to their replicas.

Migration Type	Description	Downtime?	Reversible?
Add Dimension	Add a new axis D_{i+1} to a Hypercube. All existing cells get default key.	None	Yes (remove axis — only if all cells still have default key)
Remove Dimension	Remove axis D_i . Collapses that dimension: all cells at different keys for D_i are merged (conflict resolution required).	<1 min	No — destructive. Snapshot recommended before removal.
Reorder Dimensions	Change the index order of dimensions. Primarily affects storage layout and query planning.	None	Yes
Rename Axis	Change the name or display_name of an axis. Does not affect data.	None	Yes

Migration Type	Description	Downtime?	Reversible?
Change Key Type	Change the <code>key_type</code> of an axis (e.g. String → Uuid). Requires key migration for all existing cells.	<5 min	Yes (if key format compatible)
Add Attribute Key	Register a new attribute key in the <code>AttributeKeyRegistry</code> . Existing cells gain attribute with default value.	None	Yes (if <code>no_required=true</code>)
Remove Attribute Key	Unregister an attribute key. Values for this key are stripped from all cells.	<1 min	No — destructive. <code>EventLog</code> preserves history.
Change Encoding	Migrate from Dense to Sparse or vice versa for a specific axis.	<15 min	Yes

4.4 HyperRow — The Canonical Row Entity

While any D_1 key in an `EntityAxis` is technically a "row", Hypergrid provides the `HyperRow` abstraction as the canonical first-class entity in a Hypercube. A `HyperRow` represents the full slice of the Hypercube along the D_1 axis for a given D_1 key — all its D_2 column values and all its D_3 – D_N extension cells.

```
pub struct HyperRow {
    pub row_id:      DimKey,           // D1 key for this row
    pub cube_id:     CubeId,
    pub grid_id:     GridId,

    // D2 cells — conventional "columns" for this row
    pub cells:       HashMap<D2Key, HyperCell>,

    // D3–DN extension cells — additional dimensions
    pub ext_cells:   HashMap<ExtCoord, HyperCell>,
    // ExtCoord = (D3_key, ..., DN_key) — the non-D1/D2 coordinate portion

    // Row-level metadata (shared across all cells in this row)
    pub row_attrs:   AttributeMap,     // row-level attributes
    pub vector_clock: VectorClock,
    pub version:     u64,
    pub visibility:  RowVisibility,    // Public|Tenant|Identity|Space|Private
    pub space_context: Option<SpaceContext>,
    pub graph_node_id: Option<NodeId>, // this row's node in HG-GRAPH
    pub namespace_path: Option<NamespacePath>,
    pub identity_tags: Vec<IdentityTag>,
    pub event_log_ref: EventLogRef,
}

// Row-level attribute keys are registered separately from cell-level keys:
// Row-level attrs describe the entity itself (id, name, status, owner, tags, ...)
```

```
// Cell-level attrs describe specific data points (value, format, formula, ...)
```

5. N-Dimensional Operations

5.1 Cell Addressing — Coordinate System

Every HyperCell is addressed by a DimCoordinate — an ordered tuple of N dimension keys, one per registered axis. Operations that address cells must provide a coordinate with exactly N keys, one per axis, in dimension order (D_1 first, D_N last).

```
// N=2 (conventional spreadsheet)
cell[(project_uuid, "budget_allocated")]

// N=3 (+ time dimension)
cell[(project_uuid, "budget_spent", "2026-Q2")]

// N=4 (+ geo dimension)
cell[(product_sku, "revenue", "2026-Q2", "EMEA")]

// N=5 (+ scenario dimension)
cell[(product_sku, "revenue", "2026-Q2", "EMEA", "forecast")]

// N=6 (+ org unit dimension)
cell[(employee_id, "performance_rating", "2026-H1", "US-WEST", "base_case",
"Engineering")]

// Wildcard / slice — all values on one axis
cell[(project_uuid, "budget_spent", *)]      -- all time periods for this
project+attr
cell[(*, "revenue", "2026-Q2", "EMEA")]      -- all products in EMEA in Q2 2026
```

5.2 DimSlice — N-Dimensional Filtering

A DimSlice is the generalization of a row filter to N dimensions. It defines predicates on any subset of axes, producing a sub-cube (if all non-predicate axes are retained) or a projection (if some axes are collapsed/aggregated).

DimSlice Type	Description	Example	Result
Point Slice	Exact value on one axis. Equivalent to a single row lookup (D_1) or column lookup (D_2).	$D_1 = \text{project_id_42}$	All cells for that single project across all other axes
Range Slice	Range of values on an ordered axis. Efficient on BTree or TimeSeries indexes.	D_3 (Time) BETWEEN 2026-01-01 AND 2026-06-30	All cells in H1 2026 across all rows and columns

DimSlice Type	Description	Example	Result
Set Slice	A set of specific values on any axis.	D ₄ (Region) IN ["EMEA", "APAC"]	All cells in EMEA and APAC only
Predicate Slice	A computed condition on any attribute of cells at a given axis coordinate.	D ₂ = "health_score" AND value > 80	All rows whose health_score > 80
Hierarchical Slice	Drill-down into a HierarchyAxis: select a node and all its descendants.	D ₅ (OrgUnit) UNDER "Engineering"	All cells in Engineering + all sub-units
Cross-Dim Slice	Predicates on multiple axes simultaneously. Produces the intersection sub-cube.	D ₁ = active_projects AND D ₃ = "2026-Q2"	Active projects in Q2 2026 only
DimFold	Collapse an axis by aggregating all values along that axis into a single summary.	FOLD D ₃ (Time) WITH SUM(value)	Flattened cube without time axis; values summed across all time periods
DimExpand	Expand a previously folded or defaulted axis into its full key set.	EXPAND D ₃ showing quarterly breakdown	Cube split into separate slices per quarter
NullSlice	Filter to cells where a specific axis has no explicit value (sparse cells with default key).	D ₄ IS NULL	Cells not yet assigned a geo region
ShadowSlice	Include cells from linked (Shadow) grids in results alongside owned cells.	INCLUDE SHADOW CELLS FROM grid://partner/	Own cells + shadow-linked cells from specified grid

5.3 DimFold & DimExpand — Aggregation Across Axes

DimFold and DimExpand are the N-dimensional generalizations of GROUP BY and PIVOT. They allow any axis to be collapsed into a summary (DimFold) or a previously collapsed axis to be re-expanded into its constituent keys (DimExpand).

```
-- DimFold: collapse D3 (Time) by summing revenue, for a revenue cube
SELECT D1.entity_name, D2."revenue"
FROM revenue_cube
WHERE D4.region = "EMEA"
FOLD D3 WITH SUM(value)      -- time axis collapsed, revenue summed across all
                             periods
ORDER BY D1.entity_name;

-- DimExpand: pivot D4 (Region) into separate attribute columns per region
SELECT D1.entity_name, D3.quarter,
       EXPAND D4 AS COLUMNS(value)  -- one column per region key
FROM revenue_cube
WHERE D3.quarter IN ("2026-Q1", "2026-Q2");
-- Result: entity_name | quarter | EMEA.revenue | APAC.revenue | NA.revenue | ...
```

```
-- DimFold + DimExpand (full N-dim pivot)
SELECT D1.entity_name,
       EXPAND D4 AS COLUMNS(SUM(value))
FROM revenue_cube
FOLD D3 WITH SUM(value)
GROUP BY D1.entity_name, D4.region;
-- Result: entity_name | EMEA_total | APAC_total | NA_total | ...
```

5.4 Computed Attributes Across Dimensions

ComputedAttributes (formula and AI attributes) in Hypergrid can reference cells across any dimension combination, not just within the same row. This enables cross-dimensional computations that would be impossible in a conventional spreadsheet:

Computation Pattern	Description	Formula Example	Use Case
Intra-cell	Computed from other attributes of the same cell	profit = revenue - expenses	Standard derived field
Intra-row, same dim	Computed from other D ₂ column attributes in the same D ₁ row	budget_remaining = budget_allocated - budget_spent	Row-level derived metric
Cross-D ₁ (aggregate)	Aggregate across D ₁ rows for the same D ₂ column key	avg_health = AVG(SLICE(D ₂ ="health_score"))	Column summary
Cross-D ₃ (time)	Compute a time-series aggregation across the D ₃ time axis	ytd_revenue = SUM(SLICE(D ₂ ="revenue", D ₃ BETWEEN ytd_start AND today))	YTD/rolling window KPIs
Cross-D ₄ (geo)	Aggregate across D ₄ geo axis for a specific row+column	global_sales = SUM(SLICE(D ₂ ="sales", D ₄ =*))	Geographic aggregation
Cross-dim pivot	Compute a ratio between values at different dimension key combinations	emea_pct = CELL(D ₁ ,D ₂ ,"revenue","EMEA") / global_sales	Market share / contribution
DimFold formula	Apply a formula that itself performs a DimFold as part of its computation	rolling_avg_4q = AVG(FOLD D ₃ LAST 4 QUARTERS)	Rolling window analytics
Graph-traversal	Aggregate values across graph-connected cells in HG-GRAPH	total_dependencies = SUM(GRAPH_NEIGHBORS(D ₁ =row_id, edge_type="DependsOn", attr="budget"))	Network-aware aggregation
Cross-grid join	Compute from attribute values in a different linked Grid	ext_data = JOIN(GRID("partner://grid"), D ₁ =row_id, attr="market_rate")	Cross-grid reference
Shadow cell ref	Reference a value from a shadow cell (linked from another tenant's grid)	partner_health = SHADOW_CELL(grid_id, coord, "health_score")	Cross-tenant data reference

5.5 N-Dimensional Aggregation Engine

The aggregation engine must be capable of evaluating aggregation functions across any combination of axes. The following aggregation operations are supported in any dimension combination:

Operation	Axis Types	Description	HyperQL Syntax
SUM	All numeric	Sum of value attribute across matching cells	SUM(SLICE(D ₂ ="revenue", D ₃ ="2026"))
AVG	All numeric	Mean average across matching cells	AVG(SLICE(D ₂ ="health_score"))
COUNT	All	Count of non-null cells matching the slice	COUNT(SLICE(D ₄ ="EMEA"))
COUNT_UNIQUE	All	Count of distinct values for a given attribute across a slice	COUNT_UNIQUE(SLICE(D ₂ ="owner"))
MIN / MAX	Numeric + Date	Minimum or maximum value across a slice	MIN(SLICE(D ₂ ="due_date", D ₁ =active_rows))
MEDIAN	Numeric	Median value across a slice	MEDIAN(SLICE(D ₂ ="cycle_time"))
PERCENTILE	Numeric	Arbitrary percentile: PERCENTILE(attr, p)	PERCENTILE(SLICE(D ₂ ="salary"), 0.90)
STDDEV / VAR	Numeric	Standard deviation / variance across a slice	STDDEV(SLICE(D ₂ ="lead_time"))
DISTRIBUTION	Enum / Category	Frequency distribution of enum values across a slice	DISTRIBUTION(SLICE(D ₂ ="status"))
FOLD_SUM	N-dim (any axis)	DimFold + SUM: collapse an axis by summing the value attribute	FOLD D ₃ WITH SUM(value)
FOLD_AVG	N-dim (any axis)	DimFold + AVG: collapse an axis by averaging the value attribute	FOLD D ₃ WITH AVG(value)
ROLLUP	HierarchyAxis only	Bottom-up hierarchical aggregation: leaf nodes sum to parent nodes	ROLLUP(D ₅ , attr="budget_allocated")
DRILLDOWN	HierarchyAxis only	Top-down hierarchical decomposition: show contribution of each child	DRILLDOWN(D ₅ UNDER "Engineering", attr="headcount")
MOVING_AVG	TimeAxis only	Moving window average: AVG of last N time periods for each D ₁ entity	MOVING_AVG(D ₃ , window=4, attr="revenue")
CUMULATIVE	TimeAxis only	Cumulative sum up to each time period: prefix sum along time axis	CUMULATIVE(D ₃ FROM ytd_start, attr="expenses")
GRAPH_SUM	GraphAxis + any	Sum across graph-connected nodes: aggregate neighbors in HG-GRAPH	GRAPH_SUM(D ₁ =row_id ,

Operation	Axis Types	Description	HyperQL Syntax
			edge_type="DependsOn" , attr="budget")

6. Spaces, Workspaces & Namespaces (HG-SPACE · HG-NS)

Spaces, Workspaces, and Namespaces are the three spatial organization layers of Hypergrid. They are platform-agnostic — any platform using Hypergrid inherits these primitives and can extend them for their specific domain (projects, organizations, products, research labs, events, etc.).

6.1 Space — Governed Bounded Context

A Space is a named, governed, bounded operational context within a Grid. It groups related Hypercubes, member identities, Workspaces, communication channels, and governance structures into a cohesive unit. A Space is itself a HyperRow in the system's reserved Spaces Hypercube (cube_id: "system.spaces"), giving it all the versioning, CRDT, graph, and AI capabilities of any other entity.

Space Property	Type	Description
space_id	UUID	Globally unique. Also the D ₁ key for this Space's row in the Spaces Hypercube.
space_type	SpaceType	Platform-defined enum. Examples: Personal Team Org Cooperative Community Federation Project Research Event Product Studio Tenant Custom(String)
slug	String	URL-safe unique handle within parent namespace. Used in NamespacePath.
namespace_path	NamespacePath	Full hierarchical address: hypergrid://{tenant}/{type}/{slug}/
grid_id	GridId	Parent Grid this Space belongs to.
member_roster	Vec<SpaceMember>	Members with roles. Role = PermissionTier + Space-specific role label.
cubes	Vec<CubeId>	Hypercubes scoped to this Space (Space-owned data).
shared_cubes	Vec<CubeId>	Hypercubes shared with but not owned by this Space (from other Spaces via ShadowRow links).
workspaces	Vec<WorkspaceId>	Active Workspaces within this Space.
governance_config	GovernanceConfig	Vote thresholds, quorum rules, proposal types, treasury policy, multi-sig config.
treasury_ref	Option<TreasuryRef>	Treasury account for economic Spaces. Platform connects to its financial layer.
link_node_id	NodeId	This Space's node in HG-GRAPH. All member identities link here via SpaceMembership edges.
visibility	Visibility	Private Tenant Internal Public Unlisted

Space Property	Type	Description
status	SpaceStatus	Draft Active Paused Restricted Dissolving Dissolved Archived
federation_links	Vec<FedLinkId>	Cross-Space and cross-Grid federation links.
plugin_config	Map<String,Value>	Platform-specific Space configuration registered via HG-PLUGIN.
metadata	AttributeMap	Extensible platform-defined attributes for this Space.

6.2 Space Type Taxonomy (Platform-Configurable)

The SpaceType enum is configurable per Grid deployment. The following types are provided as reference defaults — platforms register their own types via HG-PLUGIN:

Default Space Type	Description	Auto-provisioned Cubes	Governance Default
Personal	One-per-identity. Private by default. All identity's private grids, workspaces, and data.	SystemCubes + all cubes owned by this identity	Owner only
Team	Collaborative working unit. Members share cubes and workspaces.	Shared work cube + team kanban + member directory	Team lead + majority
Organization	Formal legal entity. Full governance, treasury, and member management.	Org portfolio + governance + members + finances	Org governance model (varies)
Community	Open or semi-open topical group. Content-sharing, discussion, collaboration.	Community content cube + member directory	Admin + community vote
Federation	Meta-Space linking multiple Organizations/Spaces for joint governance.	Federation portfolio + cross-space governance cube	Representative/federal
Project	Time-bounded project context. Lifecycle tied to project start/end dates.	Project tasks + timeline + resources + budget	Project owner
Research	Academic/scientific/investigative collaboration with structured outputs.	Research data cube + publications + experiments	Principal investigator
Event	Time-bounded event context. Auto-archives after event lifecycle ends.	Event schedule + attendees + session recordings	Event host
Tenant	Top-level tenancy boundary for SaaS multi-tenancy scenarios.	All tenant-owned cubes	Tenant admin
Custom	Platform-defined type registered via HG-PLUGIN with custom	Plugin-defined	Plugin-defined

Default Space Type	Description	Auto-provisioned Cubes	Governance Default
	governance, cube provisioning, and lifecycle logic.		

6.3 Workspace — Active Working Session

A Workspace is the active, personalized working context within a Space. It records the user's current arrangement of open Hypercubes and views, pinned rows, active session state, tool configuration, and AI assistant context. Workspaces are ephemeral in feel but durable in storage — all state is persisted in the system Workspaces Hypercube (cube_id: "system.workspaces").

```
pub struct Workspace {
    pub workspace_id:      UUID,
    pub workspace_type:    WorkspaceType, //
    Personal|Shared|Project|Guest|Template|AI|Archive
    pub owner_id:          IdentityId,
    pub space_id:          SpaceId,
    pub grid_id:           GridId,
    pub namespace_path:    NamespacePath,

    // Session state
    pub open_cubes:        Vec<OpenCubeRef>,
    // OpenCubeRef { cube_id, view_id, active_dim_slices, scroll_state,
    active_cell_coord }
    pub active_cube_id:    Option<CubeId>,
    pub pinned_rows:       Vec<(CubeId, D1Key)>,
    pub pinned_columns:    Vec<(CubeId, D2Key)>,
    pub recent_cells:      Vec<(CubeId, DimCoordinate, DateTime)>,

    // Active dimension context (workspace-level DimSlice defaults)
    pub dim_context:       HashMap<AxisId, DimKey>,
    // e.g. D3 (Time) = "2026-Q2" -- all cubes default to Q2 slice in this
    workspace

    // Workspace-level filters (applied across all open cubes)
    pub global_filters:    Vec<DimSlicePredicate>,

    // UI state
    pub layout_config:     WorkspaceLayout,
    pub active_board_mode: Option<BoardMode>,
    pub graph_panel_open: bool,

    // AI context
    pub ai_assistant_state: AIAssistantState,
```

```
// Session metadata
pub session:      Option<ActiveSession>,
pub metadata:     AttributeMap,
}

// Workspace-level DimContext: when a user sets D3 (Time) = "2026-Q2"
// in their workspace, ALL open cubes automatically show the Q2 slice.
// This is the N-dimensional generalization of "freeze panes" – it freezes
// a dimension at a specific key across all views simultaneously.
```

6.4 Namespace (HG-NS) — Hierarchical Addressing

The Namespace system provides globally unique, hierarchical, human-readable addresses for every entity in the Hypergrid ecosystem. The address scheme uses a URI format inspired by DNS and filesystem paths.

```
NamespacePath URI scheme:
  hypergrid://{grid_name}/{space_type}/{slug}/{entity_type}/{entity_slug}/...

Examples:
  hypergrid://acme/org/engineering/           -- Engineering Space root
  hypergrid://acme/org/engineering/cube/projects/ -- Projects Hypercube
  hypergrid://acme/org/engineering/workspace/sprint-board/ -- Workspace
  hypergrid://acme/personal/alice/           -- Alice's Personal Space
  hypergrid://acme/team/platform/cube/roadmap/row/feature-42/ -- a specific HyperRow

System-reserved addresses:
  hypergrid://{grid}/system/spaces/           -- Spaces Hypercube
  hypergrid://{grid}/system/workspaces/       -- Workspaces Hypercube
  hypergrid://{grid}/system/identities/       -- Identity registry
  hypergrid://{grid}/system/namespaces/       -- Namespace registry
  hypergrid://{grid}/system/graph/           -- Hypergraph store
  hypergrid://{grid}/system/eventlog/         -- Global EventLog
```

Namespace Operation	Description	Permission	Effect
resolve(path)	Resolve a NamespacePath to a Grid entity (GridId, Cubeld, RowId, etc.)	Viewer	Cache populated; entity returned
register(path, id)	Register a new path → entity mapping	Space owner	Entry added to NamespaceRegistry cube
rename(old, new)	Rename a path segment. Old path aliased to new.	Entity owner	Alias created; downstream refs async-updated

Namespace Operation	Description	Permission	Effect
alias(path, target)	Create an additional path that resolves to the same entity	Entity owner	Alias entry added
federate(path, peer)	Replicate a namespace entry to a federation peer grid	Federation admin	Entry pushed to peer's NamespaceRegistry
transfer(path, new_owner)	Transfer ownership of a namespace entry to another identity	Current owner	Owner updated; history preserved
list(prefix)	List all namespace entries under a given path prefix	Viewer	Paginated list of children

7. Hypergraph — Graph & Network Layer (HG-GRAPH)

Every Hypergrid deployment includes a built-in graph layer (the Hypergraph) that models relationships between any HyperCells, HyperRows, Hypercubes, and Grids as a typed, directed, weighted property graph. The Hypergraph enables cross-grid connections, dependency trees, link forests, shadow cell propagation, and network-aware computations — all from within the spreadsheet.

7.1 Hypergraph Data Model

```
pub struct Hypergraph {
  pub graph_id:      GraphId,
  pub grid_id:       GridId,
  pub nodes:         NodeStore,      // NodeId → HypergraphNode
  pub edges:         EdgeStore,      // EdgeId → HypergraphEdge
  pub subgraphs:     Vec<Subgraph>,  // Space-scoped sub-graphs
  pub link_forests:  HashMap<IdentityId, LinkForest>,
}

pub struct HypergraphNode {
  pub node_id:       NodeId,         // UUID
  pub node_type:     NodeType,       // Cell|Row|Cube|Space|Grid|External|Custom
  pub entity_ref:    EntityRef,      // reference to the entity this node
  represents
  // EntityRef = CellRef|RowRef|CubeRef|SpaceRef|GridRef|ExternalRef
  pub attributes:    AttributeMap,   // node-level N-attributes (reuses HG-CELL
  model)
  pub namespace_path: Option<NamespacePath>,
  pub visibility:    NodeVisibility,
  pub identity_tag:  Option<IdentityTag>,
}

pub struct HypergraphEdge {
  pub edge_id:       EdgeId,
  pub from_node:     NodeId,
  pub to_node:       NodeId,
  pub edge_type:     EdgeType,       // see §7.2
  pub direction:     EdgeDirection,  // Directed|Bidirectional
  pub weight:        f64,            // 0.0-1.0
  pub attributes:    AttributeMap,   // edge-level N-attributes
  pub visibility:    EdgeVisibility,
  pub consent_status: ConsentStatus, // Pending|Accepted|Declined|Revoked
  pub space_scope:   Option<SpaceId>,
  pub shadow_cell_id: Option<ShadowCellId>, // shadow cell created by this edge
  pub expires_at:    Option<DateTime<Utc>>, // for time-limited edges
}
```

```

    pub created_at:      DateTime<Utc>,
}

```

7.2 Edge Type Registry

Edge Type	Source Node	Target Node	Description	Shadow Cell?
Hierarchy	Any	Any	Parent-child containment. Used for drill-down and rollup along any axis.	No
Dependency	Row/Cell	Row/Cell	One entity depends on another (blocking relationship). Cross-grid supported.	Optional
Association	Any	Any	Soft lateral association. Non-blocking reference between entities.	No
Contains	Space/Cube	Cube/Row	Ownership containment: this Space/Cube contains this entity.	No
Derives	Row	Row	One row is derived from or is a version/fork of another.	Optional
CrossGridLink	Row/Cell	Row/Cell	Inter-grid connection: the core link network edge. Enables link forests.	Yes
ShadowOf	ShadowRow	Source Row	The inverse of CrossGridLink: marks a row as a shadow of a row in another grid.	N/A (IS shadow)
SpaceMembership	Identity	Space	A tenant/identity is a member of a Space.	No
Collaborates	Identity	Row/Cube	Bidirectional collaboration on a shared entity.	Yes
References	Cell/Row	Cell/Row	Citation or reference. Lighter than Dependency.	No
FederationPeer	Grid	Grid	Two Grids are federated for CRDT sync. All replicated entities share edges here.	N/A (system)
NamespaceAlias	Namespace	Namespace	One namespace path is an alias for another.	No
ComputedFrom	Cell	Cell(s)	This cell's attribute value was computed from the target cell(s).	No
SubscribesTo	Identity	Row/Cube/Space	Identity receives change notifications for this entity.	Optional
InvestedIn	Identity	Row/Space	Economic investment relationship. Platform-specific semantics.	Yes

Edge Type	Source Node	Target Node	Description	Shadow Cell?
Custom(String)	Any	Any	Platform-defined edge type registered via HG-PLUGIN.	Plugin-defined

7.3 Shadow Cell Protocol

The Shadow Cell protocol is the generalization of Hypergrid's ShadowRow pattern to the N-dimensional model. When a CrossGridLink edge is established between a cell/row in Grid A and a cell/row in Grid B, a ShadowCell (or ShadowRow) is provisioned in Grid B — a read-only reflection of the linked entity from Grid A.

Phase	Description	N-Dim Extension
Link Request	Entity A creates a CrossGridLink to Entity B. If consent required, invitation sent to Entity B's owner.	The link can target a specific DimCoordinate, a full HyperRow, an entire Hypercube, or a DimSlice.
Visibility Config	Entity B configures which attribute keys are visible in the ShadowCell, and optionally which attributes Entity A can write back.	In N-dim: can also configure which dimension keys are shadowed (e.g. only share time periods $D_3 \geq \text{"2026"}$).
Shadow Provisioning	ShadowCell (or ShadowRow) created in Grid A's grid, registered in HG-GRAPH with ShadowOf edge.	ShadowCell carries a dim_scope: which dimensional slices of the source are reflected.
Attribute Sync	When source entity's visible attributes change, ShadowCell receives delta via Kafka. Mirror attributes updated in Cell Cache.	Only attribute keys in visible_attrs and dimension keys in dim_scope are synced.
Write-back	If write_back_attrs configured, Grid A's edits to those attributes are propagated back to the source as cross-grid CrdtOps.	Write-back can be scoped to specific dimension keys (e.g. only write-back comments, not financial data).
Disconnection	When CrossGridLink is deleted, ShadowCell marked Disconnected. Last known values preserved in EventLog.	All N-dim slices of the ShadowCell disconnected simultaneously.

7.4 Link Forest & Tree Traversal

Traversal Mode	Algorithm	Depth	N-Dim Extension
Ego Graph	BFS from root node; depth ≤ 2 ; all cross-grid edges	2	Filter by edge type and by dimension scope (e.g. only show links where $D_3 = \text{current quarter}$)
Link Forest	BFS from all nodes owned by an identity; union of trees	3	Forest scoped to a DimSlice: shows only links relevant to current cube+dim context

Traversal Mode	Algorithm	Depth	N-Dim Extension
Dependency Tree	Topological sort on Dependency edges; cycle-safe	unlimited	Full N-dim coordinates shown per node; critical path highlights cross-dim bottlenecks
Component Tree	DFS from a single row/cell node; all in/out edges	5	N-dim attribute panel shown for each node; filter by attribute key
Space Subgraph	All edges between members of a Space	4	Slice to show only edges involving a specific dimension (e.g. geo=EMEA connections)
Shortest Path	Dijkstra on weighted Hypergraph; edge.weight as cost	computed	Edge weights can incorporate N-dim attribute values (e.g. D ₃ -weighted time proximity)
Cluster Detection	Louvain on undirected projection of full link graph	full	Cluster assignments exposed as an additional computed attribute on HyperRow nodes
Centrality	PageRank / betweenness (sampled approximation for scale)	full	Centrality stored as AI-computed attribute on each node row

8. Multi-Tenancy & Identity (HG-ID)

HG-ID provides a platform-agnostic multi-tenancy and multi-identity model. "Tenant" is the general term for the Sovereign Entity of a Kogi-style system — it can be a user, organization, research group, or machine agent. Every tenant may have multiple identities, each with its own namespace, visibility configuration, and partition of the shared grid.

8.1 Tenant & Identity Hierarchy

Level	Concept	Definition	Grid Mapping
L0	SovereignTenant	The real-world entity (person, org, agent, service) that owns a root set of Hypercubes and Spaces in a Grid.	One SovereignTenant = one root namespace root + one Personal Space
L1	Account	A credential set (API key, OAuth, passkey) granting authenticated access to a SovereignTenant's grid.	Multiple accounts may share one SovereignTenant. Each has an access scope (cube + attribute + dim restrictions).
L2	Identity	A named, purpose-specific @handle with its own display name, bio, and public profile.	Each Identity is a DimensionKey in the system.identities cube + a HypergraphNode.
L3	TenantPartition	A logical partition of a SovereignTenant's grid, scoped to one Identity. Rows tagged to a partition via identity_tags.	NOT a physical copy. OR-Set of identity_tag attribute keys on HyperRows determines partition membership.
L4	VisibilityMask	Rules specifying which rows, cells, and attribute keys are visible to which observer type.	Applied at query time: DimSlice predicate injected based on caller's identity + relationship to data owner.
L5	SplitPolicy	Governance rules on cross-partition data access. Enforced by PolicyEngine on every CrdtOperation crossing partition boundaries.	PolicyEngine.evaluate() called on every cross-partition write. HardIsolated partitions have field-level encryption.

8.2 TenantPartition Data Model

```
pub struct TenantPartition {
    pub partition_id:      UUID,
    pub identity_handle:   String,           // @handle — unique within
    SovereignTenant
    pub sovereign_id:     SovereignTenantId,
    pub display_name:     String,
    pub isolation_level:   IsolationLevel,   // None | Soft | Hard
    pub visibility_mask:   VisibilityMask,
    pub identity_tag:     IdentityTag,       // tag applied to rows in this
    partition
    pub linked_accounts:   Vec<AccountId>,
    pub linked_spaces:    Vec<SpaceId>,
```

```

    pub graph_node_id:      NodeId,          // this identity's node in HG-GRAPH
    pub namespace_path:     NamespacePath,   //
hypergrid://{grid}/identity/{handle}/
    pub split_policy_id:    Option<PolicyId>,
    pub public_cube_view_id: ViewId,         // public profile view
    pub follower_view_id:   ViewId,
    pub connection_view_id: ViewId,
}

pub struct VisibilityMask {
    pub public_row_filter:   DimSlicePredicate, // which rows are public
    pub follower_row_filter: DimSlicePredicate,
    pub connection_filter:  DimSlicePredicate,
    pub public_attrs:       Vec<AttributeKey>, // which attrs visible to public
    pub follower_attrs:     Vec<AttributeKey>,
    pub connection_attrs:   Vec<AttributeKey>,
    pub owner_only_attrs:   Vec<AttributeKey>, // never visible to non-owner
    pub custom_rules:       Vec<VisibilityRule>,
    // VisibilityRule: (DimSlicePredicate, AttrSet) - context-dependent visibility
}

```

8.3 N-Dimensional Visibility — DimSlice-Based Visibility Masks

In Hypergrid's N-dimensional model, visibility masks are not just row-level filters — they are full N-dimensional DimSlice predicates. This enables sophisticated visibility rules that vary by any combination of dimension axes:

Visibility Pattern	DimSlice Predicate	Description
Row-level (conventional)	$D_1 = \{\text{owned_by_me}\}$ OR $D_1.\text{visibility} = \text{Public}$	Standard row-level visibility. Works identically to 2D spreadsheet access control.
Column-level	$D_2 \text{ IN } \{\text{public_attrs}\}$ (for non-owner callers)	Certain column keys hidden from specific observer types.
Time-scoped	$D_3 \geq \{\text{public_from_date}\}$ (TimeAxis)	Historical data visible to public only from a specific date onward. Older data owner-only.
Geo-scoped	$D_4 \text{ IN } \{\text{permitted_regions}\}$ (GeoAxis)	Geographic data access: a researcher can only see data for their approved countries.
Scenario-scoped	$D_5 \text{ IN } \{\text{"base_case"}\}$ (non-owner excludes forecast/pessimistic)	Scenario axes: public sees only base case; internal users see all scenarios.
Org-unit-scoped	$D_6 = \{\text{caller's_org_unit}\}$ OR $D_6 = \text{"cross-unit-approved"}$	Users see only their own org unit's data plus explicitly shared cross-unit rows.
N-dim combined	$D_1 = \text{active}$ AND $D_3 \geq \text{2026-Q1}$ AND $D_4 \text{ IN } \{\text{allowed_regions}\}$	Full N-dim visibility intersection: multiple axes restrict simultaneously.
Attribute-conditional	WHEN $D_2 = \text{"salary"}$ THEN visibility=Owner; WHEN $D_2 = \text{"name"}$ THEN visibility=Public	Attribute-conditional: different attrs of the same row/cell have different visibility rules.

Visibility Pattern	DimSlice Predicate	Description
Graph-traversal-based	D_i IN (GRAPH_NEIGHBORS(my_node, edge="Collaborates"))	Social/network visibility: can see rows of direct collaborators but not the wider network.

9. View Engine (HG-VIEW)

The View Engine transforms the raw N-dimensional cell store into shaped, filtered, sorted, grouped, and visually rendered presentations. A HypercubeView is the primary user-facing artifact: a fully described, shareable, versionable configuration that projects a sub-space of the Hypercube onto a specific rendering surface.

9.1 HypercubeView Data Model

```
pub struct HypercubeView {
    pub view_id:      ViewId,
    pub cube_id:      CubeId,
    pub name:         String,
    pub namespace_path: NamespacePath,

    // Dimensionality configuration
    pub primary_axis:  AxisId,      // which axis maps to "rows" in 2D
    rendering
    pub secondary_axis: AxisId,      // which axis maps to "columns" in 2D
    rendering
    pub dim_slices:    Vec<DimSlice>, // predicates on all other axes
    pub dim_folds:     Vec<DimFold>,  // axes collapsed by aggregation
    pub dim_expands:   Vec<DimExpand>, // axes expanded as pivot columns

    // Column/row configuration
    pub attr_schema:   Vec<AttrColumnDef>, // which attribute keys shown +
    order
    pub pinned_attrs:  Vec<AttributeKey>,  // always-visible attributes (left
    freeze)
    pub hidden_attrs:  Vec<AttributeKey>,  // hidden from this view
    pub col_widths:    HashMap<AttributeKey, u32>,

    // Filtering / Sorting / Grouping
    pub filters:       Vec<DimSlicePredicate>,
    pub sorts:         Vec<ViewSort>,
    pub groups:        Vec<ViewGroup>,

    // Rendering mode
    pub render_mode:   RenderMode,
    // Grid2D | Kanban | Gantt | Calendar | NetworkGraph | Treemap |
    // Timeline | HierarchyTree | PivotTable | HeatMap | Matrix |
    // LinkForest | N-DimExplorer | Custom(PluginId)

    // Augmentation
    pub highlight_rules: Vec<HighlightRule>,
    pub summary_row:     Option<SummaryRowConfig>,
    pub shadow_policy:   ShadowCellPolicy, // Include|Exclude|ShadowOnly
}
```

```

pub identity_filter: Option<IdentityTag>,
pub space_filter:   Option<SpaceId>,
pub graph_overlay:  bool,
pub ai_overlay:     bool,

// Sharing
pub visibility:     ViewVisibility, //
Private|Shared|SpaceShared|Public|Template
pub created_by:     IdentityId,
pub metadata:       AttributeMap,
}

```

9.2 Axis Remapping — Any Dimension Can Be "Rows" or "Columns"

One of the most powerful features of the HypercubeView is axis remapping: the user can choose which dimension maps to the row axis and which maps to the column axis for a given view, without changing the underlying data. This enables radically different perspectives on the same cube:

View Name	primary_axis (rows)	secondary_axis (cols)	Description
Standard	D ₁ (Entity)	D ₂ (Property)	Conventional spreadsheet: entities as rows, properties as columns
Transposed	D ₂ (Property)	D ₁ (Entity)	Properties as rows, entities as columns — useful for attribute-comparison views
Time Series	D ₃ (Time)	D ₂ (Property)	Time periods as rows, properties as columns — timeline/history view
Geographic	D ₄ (Region)	D ₂ (Property)	Regions as rows, properties as columns — geo breakdown view
Scenario Compare	D ₅ (Scenario)	D ₁ (Entity)	Scenarios as rows, entities as columns — plan-vs-actual comparison
Cross-dim Matrix	D ₃ (Time)	D ₄ (Region)	Time × Geography matrix with D ₁ -aggregated values — heatmap ready
Entity × Time	D ₁ (Entity)	D ₃ (Time)	Entities as rows, time periods as columns — standard time-series pivot
Org × Scenario	D ₆ (OrgUnit)	D ₅ (Scenario)	Org units as rows, scenarios as columns — planning comparison matrix

9.3 Render Modes

Render Mode	Description	Primary/Secondary Axes	Best For
Grid2D	Standard 2D spreadsheet grid. Rows × Columns with cell values.	Any two axes	General data viewing, editing, and bulk operations
PivotTable	Cross-tabulation of two axes with aggregate values. DimFold + DimExpand applied.	Any two + fold/expand	Multi-dimensional summaries; cross-dim analysis
HeatMap	Color-coded matrix: rows × columns with value-to-color mapping.	Any two numeric axes	Pattern detection; anomaly identification; risk matrices
Matrix	Square matrix for relationship visualization (dependency matrix, correlation matrix).	Any two (same type)	Dependencies; correlations; cross-entity relationships
Kanban	Swimlane columns = enum axis values; cards = row entities with key attributes.	D ₁ (entity), D ₂ /D ₃ (status)	Workflow management; status-driven work
Agile	Sprint-scoped Kanban with story points, burndown overlay, velocity tracking.	D ₁ (tasks), sprint dim	Software development; agile project management
Gantt	Horizontal bars from start to end date; dependencies as arrows; critical path.	D ₁ (entity), D ₃ (time)	Project planning; timeline visualization
Calendar	Day/week/month grid rendering by date attribute.	D ₃ (time), D ₁ (entity)	Scheduling; event planning; deadline management
Timeline	Swimlane rows; bars per entity across time axis.	D ₁ (entity), D ₃ (time)	Roadmaps; strategic planning; phased rollouts
HierarchyTree	Collapsible tree rendered along a HierarchyAxis.	D ₁ with hierarchy	Org charts; file systems; product taxonomies
Treemap	Hierarchical tiles sized by numeric attribute; color by score attribute.	D ₁ hierarchy, D ₂ (size)	Budget composition; portfolio breakdown; category distribution
NetworkGraph	Force-directed graph: nodes = row entities; edges = HG-GRAPH edges. Color by attribute.	D ₁ (entities) + edges	Dependency visualization; link network exploration
LinkForest	KLNK-style tree rendering of cross-grid links. Identity-colored branches.	D ₁ + CrossGridLink edges	Inter-grid connection visualization; partner network
N-DimExplorer	Interactive N-dimensional browser: choose primary/secondary axes, sliders for all other axes. Pan/zoom.	All N axes user-selectable	Deep exploratory N-dim analysis; data science
Custom	Platform-defined render mode registered via HG-PLUGIN.	Plugin-defined	Domain-specific visualizations (maps, genomic

Render Mode	Description	Primary/Secondary Axes	Best For
			plots, circuit diagrams)

10. Distributed Consistency (HG-CRDT)

Hypergrid is distributed-first: every Grid is designed for multi-node, multi-device, offline-capable concurrent editing. Consistency is maintained by a generalized N-dimensional CRDT that applies appropriate merge semantics per attribute type, per axis type, and per governance policy.

10.1 N-Dimensional CRDT Model

Hypergrid's CRDT operates at three granularity levels:

CRDT Level	Applies To	Semantics	Merge Strategy
Grid-level	Full Grid CRDT sync across federation peers	Gossip protocol: each peer maintains a vector clock per grid; delta sync on heartbeat	VectorClock merge: element-wise max; delta replay in causal order
Hypercube-level	Schema: dimension additions, attribute registrations	SchemaEvent log: ordered, idempotent. Schema merges are non-conflicting if additive.	All additive schema changes commute. Destructive changes require governance gate.
Cell-attribute-level	Individual cell attribute values	Per-attribute CRDT semantics (see §10.2). Each attribute key in the AttributeKeyRegistry has independently configured CRDT semantics.	Per-attribute: LWW OR-Set Counter Lattice

10.2 Per-Attribute CRDT Semantics

CRDT Type	Attribute Types	Merge Rule	Conflict Behavior
LWW (Last-Write-Wins)	Text, Number, Enum, Date, Bool, Relation	Highest VectorClock timestamp wins. Tiebreak: ActorId lexicographic sort.	Losing write stored in ConflictRecord; surfaced to users for review. Oba notifies affected actors.
OR-Set	MultiEnum, Tag, MultiRelation, MultiUser	All concurrent Adds survive. Removes tagged: only the exact tagged entry removed.	No conflict for concurrent Adds. Remove vs concurrent Add: both survive (Add wins per OR-Set semantics).
GrowOnly Counter	view_count, follower_count, analytics counters	Sum all increment operations. Never decrements.	Always commutes. No conflict possible.
PN-Counter	budget_spent, allocated_units	Sum of (positive_ops - negative_ops) per node.	Commutes across all concurrent increments/decrements.

CRDT Type	Attribute Types	Merge Rule	Conflict Behavior
Max Register	version, sequence_number, generation	Highest numeric value wins.	Always resolves to maximum; no human review needed.
Min Register	earliest_deadline, creation_timestamp	Lowest value wins.	Always resolves to minimum; no human review needed.
Status Lattice	status, state, lifecycle_stage	Merge to least upper bound in the configured partial order. No raw LWW for status.	Governance gate: lattice transitions that require approval are flagged for GovernanceWorkflow before applying.
Append-only Log	EventLog, comment_thread, change_history	All appends commute. Causally ordered by VectorClock.	No conflict: all appends survive. Causal order guaranteed.
JSON CRDT	Json, plugin-defined complex attrs	Deep-merge of JSON objects: recursive LWW per leaf key.	Key-level conflicts use LWW; structural conflicts (key type change) surfaced as ConflictRecord.
Custom (Plugin)	Custom attribute types	Plugin provides merge(a: AttrValue, b: AttrValue, ctx: MergeContext) → AttrValue.	Plugin-defined conflict resolution. Plugin may surface ConflictRecord for human review.

10.3 N-Dimensional Conflict Scenarios

Scenario	Description	Resolution
Concurrent cell value writes (same coord)	Two nodes write different values to the same ($D_1k, D_2k, \dots, DNk$) coordinate's "value" attribute simultaneously.	LWW: higher VectorClock timestamp wins. Losing value in ConflictRecord.
Concurrent dimension key add + remove	Node A adds D_3 key "2026-Q3"; Node B removes "2026-Q3" concurrently.	OR-Set semantics: Add wins over concurrent Remove. Key retained.
Concurrent schema changes	Two nodes add different custom axes D_3 simultaneously.	Additive schema changes commute. Both axes added. dim_index assigned by timestamp order.
Concurrent status transitions (lattice)	Node A: status Active → Paused. Node B: status Active → Completed. Both concurrent.	Lattice join: Completed dominates Paused (further in lifecycle). Status = Completed. ConflictRecord for visibility.
Cross-partition write (identity boundary)	Actor from Identity A writes a cell attribute on a HyperRow tagged to Identity B (HardIsolated).	PolicyEngine: DENY. Operation rejected at gateway. CrossPartitionViolationEvent appended to audit log.
DimSlice predicate conflict (view filter)	Concurrent view edits change incompatible DimSlice predicates for the same view.	LWW per ViewDefinition attribute. Latest full ViewDefinition wins. Previous view preserved in ViewHistory.

Scenario	Description	Resolution
Shadow cell sync vs local edit	Grid A's ShadowCell has a value; Grid B (source) updates it; simultaneously Grid A's user edits the shadow cell locally.	Shadow cells are read-only for non-write-back attributes. Local edit rejected. User shown "shadow cell — edit at source" message.
Concurrent axis rename	Two federation peers rename the same axis with different names simultaneously.	LWW: higher VectorClock timestamp wins for axis.name. Alias created for the losing name.
Concurrent federation peer add	Two Grid admins add different federation peers simultaneously.	OR-Set: both federation peers added. VectorClock extended for both.

10.4 Federation Protocol

```
// Federation sync lifecycle (same for Grid-to-Grid)
```

1. DISCOVERY: Register peer. POST /federation/peers { peer_url, trust_level }
→ Mutual challenge-response auth. Grid IDs exchanged. Peer record stored.
2. HANDSHAKE: Exchange current VectorClock state + schema hashes.
→ Identify: (a) schema delta (dim/attr additions), (b) event delta (cell mutations)
3. SCHEMA SYNC: Replay SchemaEvents missing from peer (additive only).
→ Peer applies dimension additions, attribute registrations idempotently.
4. CELL DELTA SYNC: Compute set of CrdtOperations since peer's last VectorClock.
→ Batch into chunks of max 10,000 ops. Compress (Zstd). Send to peer.
→ Peer applies CrdtOperation stream via apply_crdt_op() in causal order.
5. CONFLICT SURFACE: Conflicts detected during apply are sent back to origin.
→ Origin surfaces ConflictRecord to relevant actors via HG-AI anomaly + notification.
6. NAMESPACE SYNC: Propagate NamespaceRegistry entries for federated entities.
7. HEARTBEAT: Every 30s. Exchange VectorClock only.
→ If divergence detected: trigger full delta sync.
8. SHADOW SYNC: Ongoing. Kafka subscription per shadow-linked entity.
→ Cell attribute delta pushed to subscribing grids via shadow_sync topic.

11. Computation Engine (HG-COMP)

11.1 Two-Tier Architecture

Tier	Description	Latency	Technology
Tier 1 — Synchronous	Arithmetic and structural ComputedAttributes: derived from other cell attributes in the same HyperRow, simple aggregations over a DimSlice, formula expressions without AI/ML.	<1ms per cell	Rust ComputedAttrEvaluator, embedded in cell query pipeline
Tier 2 — Asynchronous	AI/ML-driven signals: anomaly detection, health scoring, classification, prediction, recommendation, entity resolution.	50ms–30s (model-depe ndent)	Pluggable AIEngineAdapter + CellCache (Redis) + WritebackProtocol

11.2 Formula Language — HyperQL Expressions

Category	Functions	N-Dim Extensions
Arithmetic	+ − × ÷ % ^ ABS ROUND CEIL FLOOR MIN MAX	Operands can be any DimCoordinate reference: CELL(D ₁ =id, D ₂ ="revenue", D ₃ ="2026-Q2")
Comparison	= != > < >= <=	Compare across dim keys: CELL(D ₃ ="2026-Q2", D ₂ =attr) > CELL(D ₃ ="2026-Q1", D ₂ =attr)
Logical	AND OR NOT IF(cond,t,f) SWITCH COALESCE	
String	CONCAT UPPER LOWER LEN TRIM LEFT RIGHT SUBSTR CONTAINS STARTS	
Date	TODAY NOW YEAR MONTH QUARTER DAYS_BETWEEN DATE_ADD DATE_FORMAT	Works naturally on TimeAxis keys
Aggregation	SUM AVG COUNT MIN MAX MEDIAN PERCENTILE STDDEV DISTRIBUTION	Over any DimSlice: SUM(SLICE(D ₃ ="2026", D ₄ ="EMEA", D ₂ ="revenue"))
DimFold	FOLD(axis_id, agg_fn, attr)	Collapse any axis: FOLD(time_axis, SUM, "revenue")
DimSlice	SLICE(dim_predicates...) → set of cells	Any combination of axis predicates: SLICE(D ₁ =active, D ₃ >= ytd_start)
Graph	GRAPH_NEIGHBORS GRAPH_PATH GRAPH_DIST GRAPH_CENTRALITY	Traverses HG-GRAPH: GRAPH_NEIGHBORS(D ₁ =row_id, "DependsOn", attr="budget_allocated")
Shadow	SHADOW_CELL(grid_id, coord, attr)	Reference a mirror-attribute from a shadow cell in a linked grid
AI Engine	AI_SCORE(model_id, attrs...) AI_CLASSIFY AI_PREDICT AI_ANOMALY	Inject Tier-2 AI signals as formula operands. Cached per model TTL.

Category	Functions	N-Dim Extensions
Namespace	NS_RESOLVE(path) NS_PATH(entity_ref) NS_SCOPE(entity_ref)	Navigate by namespace address
Space	SPACE_NAME(space_id) SPACE_MEMBER_COUNT(space_id) IN_SPACE(space_id)	
Lookup	RELATED(rel_attr, target_attr) LOOKUP(key, cube_id, key_attr, value_attr)	Cross-cube join: RELATED(customer_id_attr, "customer_name") from CRM cube
Identity	IDENTITY_TAG() IS_PARTITION(tag) VISIBLE_TO(obs_type)	VisibilityMask-aware formula expressions
Time	AS_OF(timestamp, formula)	Time-travel: evaluate formula against historical cell state

11.3 Pluggable AI Engine (HG-AI)

The AI Engine in Hypergrid is fully pluggable — any platform can connect their own ML/AI service to provide Tier-2 computed attributes. The HG-AI module defines the AIEngineAdapter interface:

```
pub trait AIEngineAdapter {
    // Register a computed attribute model
    fn register_model(&self, model: AIModel) -> Result<ModelId, AIError>;

    // Compute a batch of cell attribute values
    fn compute_batch(
        &self,
        requests: Vec<ComputeRequest>,
        context: &ComputeContext,
    ) -> Result<Vec<ComputeResult>, AIError>;

    // Detect anomalies across a DimSlice
    fn detect_anomalies(
        &self,
        slice: &DimSlice,
        attr: &AttributeKey,
        sensitivity: f32,
    ) -> Result<Vec<AnomalySignal>, AIError>;

    // Translate natural language to HyperQL
    fn nl_to_query(
        &self,
        natural_language: &str,
        cube_context: &CubeContext,
    ) -> Result<HyperQLQuery, AIError>;
}
```

```
// Writeback: accept AI-computed values and write to cells
fn writeback_protocol(&self) -> WritebackProtocolRef;
}

// Built-in reference implementation: OpenAI-compatible adapter
// Platform can replace with local LLM, custom ML service, or any REST/gRPC AI API
```

12. HyperQL — N-Dimensional Query Language (HG-QL)

HyperQL is the N-dimensional query language of Hypergrid. It is a SQL-inspired language typed to the N-dimensional cell model. All DimSlice, DimFold, DimExpand, time-travel, shadow-cell inclusion, cross-grid join, and graph-traversal operations are expressible as first-class HyperQL constructs.

12.1 Full Grammar (EBNF)

```

query      ::= SELECT sel FROM source where? fold? expand? group? order? limit?
options?

sel        ::= "*" | sel_item ("," sel_item)*
sel_item   ::= attr_key | agg_expr | formula_expr | dim_key_ref | "EXPAND" axis_id

source     ::= cube_ref | grid_ref | "SHADOW CUBES" | "(" query ")"
cube_ref   ::= cube_id | namespace_path
grid_ref   ::= "GRID(" string_lit ")"

where      ::= "WHERE" condition
condition  ::= predicate
              | "(" condition "AND" condition ")"
              | "(" condition "OR"  condition ")"
              | "NOT" condition

predicate  ::= dim_pred | attr_pred | graph_pred | shadow_pred
dim_pred   ::= "D" uint "=" dim_key
              | "D" uint "IN" "[" dim_key ("," dim_key)* "]"
              | "D" uint "BETWEEN" dim_key "AND" dim_key
              | "D" uint "UNDER" dim_key           -- HierarchyAxis drill-down
              | "D" uint "IS NULL"
              | "D" uint "IS NOT NULL"
attr_pred  ::= attr_key op val
              | attr_key "IS EMPTY" | "IS NOT EMPTY"
              | attr_key "INCLUDES" val
              | attr_key "HAS_SEVERITY" severity
graph_pred ::= "CONNECTED_TO(" node_id ("," edge_type ")"
              | "GRAPH_DISTANCE(" node_id ")" op uint
shadow_pred ::= "IS SHADOW" | "IS NOT SHADOW"

fold       ::= "FOLD" axis_id "WITH" agg_fn "(" attr_key ")"
expand     ::= "EXPAND" axis_id "AS COLUMNS(" sel_item ")"
group      ::= "GROUP BY" groupby_item ("," groupby_item)* rollup?
rollup     ::= ", ROLLUP(" axis_id ")"

```

```

order      ::= "ORDER BY" order_item ("," order_item)*
order_item ::= attr_key | dim_key_ref | agg_expr
            | ("ASC" | "DESC") ("NULLS FIRST" | "NULLS LAST")?
limit      ::= "LIMIT" uint ("OFFSET" uint)?

options    ::= option+
option     ::= identity_scope | space_scope | shadow_opt | as_of_opt | ns_opt |
graph_opt

identity_scope ::= "IDENTITY" string_lit
space_scope    ::= "IN SPACE" string_lit
shadow_opt     ::= "INCLUDE SHADOW CELLS"
                | "SHADOW FROM" string_lit    -- specific grid/tenant
                | "SHADOW ONLY"
as_of_opt      ::= "AS_OF" timestamp
ns_opt         ::= "UNDER" namespace_path    -- namespace subtree filter
graph_opt      ::= "TRAVERSE GRAPH" edge_type_list depth_spec

```

12.2 HyperQL Examples

```

-- 1. Basic 2D query (conventional spreadsheet equivalent)
SELECT name, status, budget_allocated, budget_spent, health_score
FROM projects_cube
WHERE D2.status = "Active"
ORDER BY health_score DESC
LIMIT 50;

-- 2. Time-sliced query (N=3)
SELECT D1.entity_name, D2.revenue, D2.expenses, D2.profit
FROM financials_cube
WHERE D3 BETWEEN "2026-01-01" AND "2026-06-30"
      AND D1.status = "Active"
ORDER BY D2.revenue DESC;

-- 3. Geographic + time DimFold
SELECT D1.product_name,
       SUM(D2.revenue) AS total_revenue_emea
FROM sales_cube
WHERE D4 = "EMEA"
FOLD D3 WITH SUM(revenue)    -- collapse time: sum all periods
GROUP BY D1.product_name
ORDER BY total_revenue_emea DESC;

-- 4. Full N-dim pivot (time × region)
SELECT D1.product_name,
       EXPAND D4 AS COLUMNS(SUM(revenue))
FROM sales_cube
FOLD D3 WITH SUM(revenue)
GROUP BY D1.product_name, D4.region;

```



```

-- Result: product_name | EMEA_revenue | APAC_revenue | NA_revenue | LATAM_revenue

-- 5. Hierarchy drilldown (HierarchyAxis)
SELECT D6.org_unit, SUM(budget_allocated), AVG(health_score)
FROM projects_cube
WHERE D6 UNDER "Engineering"      -- Engineering + all sub-units
GROUP BY D6.org_unit, ROLLUP(D6);

-- 6. Graph-traversal query
SELECT D1.project_name, D2.budget_allocated,
       GRAPH_NEIGHBORS(D1, "DependsOn", attr="project_name") AS dependencies
FROM projects_cube
WHERE D2.status = "Active"
TRAVERSE GRAPH "DependsOn" DEPTH 3;

-- 7. Shadow cell inclusion
SELECT D1.entity_name, D2.status, D2.health_score,
       shadow_source_grid AS source_grid
FROM projects_cube
WHERE D2.status = "Active"
INCLUDE SHADOW CELLS
ORDER BY D2.health_score DESC;

-- 8. Time-travel query
SELECT D1.entity_name, D2.status, D2.budget_remaining
FROM projects_cube
AS_OF "2026-01-01T00:00:00Z";      -- state as of Jan 1 2026

-- 9. Cross-grid join
SELECT p.project_name, p.budget_allocated,
       c.customer_name, c.mrr
FROM projects_cube AS p
JOIN GRID("crm://acme/cube/customers") AS c
  ON p.customer_ref = c.D1;

-- 10. Namespace-scoped query
SELECT D1.entity_name, D2.status
FROM *
UNDER "hypergrid://acme/org/engineering/"
WHERE D2.status = "Active";

-- 11. Identity-partitioned query
SELECT D1.entity_name, D2.income, D2.benefit_balance
FROM personal_finance_cube
IDENTITY "@alice-freelance"        -- only rows in this identity partition
ORDER BY D2.income DESC;

-- 12. Multi-dim scenario comparison

```

```
SELECT D1.product_name,  
       CELL(D1, D2="revenue", D3="2026-Q2", D5="base_case") AS base_revenue,  
       CELL(D1, D2="revenue", D3="2026-Q2", D5="optimistic") AS opt_revenue,  
       CELL(D1, D2="revenue", D3="2026-Q2", D5="pessimistic") AS pess_revenue  
FROM planning_cube  
ORDER BY base_revenue DESC;
```

13. Persistence & Storage Architecture

13.1 Storage Layer Design

Store	Technology	Responsibility	Encoding	Access
Primary Cell Store	PostgreSQL 16 + JSONB	HyperCell data for Dense/Hybrid cubes. Schema: (grid_id, cube_id, d1_key, d2_key, ..., dn_key, attributes JSONB).	HASH partitioned by (grid_id, cube_id)	Read-heavy JSONB-compos
Sparse Cell Store	PostgreSQL 16	HyperCell data for Sparse cubes. Only non-null cells stored. Full N-dim coordinate as composite PK.	Composite PK + GIN on attributes	O(log n) scan via
Schema Registry	PostgreSQL 16	DimensionAxis definitions, AttributeKeyRegistry, CubeDefinitions, GridDefinitions.	Normalized relational schema	Read on cached
Event Log	PostgreSQL 16 (append-only, monthly partitioned)	Immutable log: every cell mutation, schema change, governance action, graph event.	Partitioned by month; append-only	Append mutation audit
CRDT Buffer	Redis Cluster	In-flight CrdtLog operations; VectorClock cache; hot-path LWW cell cache.	Hash by (grid_id, cube_id, coord_hash)	Sub-ms expiry; a
Cell Attribute Cache	Redis Cluster	Computed and AI attribute values. TTL per model. Invalidated on cell mutation or schema event.	Key: cell_cache:{grid}:{cube}:{coord_hash}:{attr_key}	O(1) read invalidat
Graph Store	PostgreSQL 16 (+ optional Neo4j for scale)	HypergraphNode and HypergraphEdge tables. Cross-grid link edges. Shadow cell mapping.	B-tree + GIN on edge_type, node attributes	Graph topology
Space/Workspace Store	PostgreSQL 16	Space, Workspace, SpaceMember, GovernanceProposal, WorkspaceSession records.	Normalized relational	Read on events
Namespace Registry	PostgreSQL 16 + Redis	NamespacePath → entity mapping. Resolution cache in Redis (300s TTL).	BTree index on path; Redis cache	Every re Redis hi
Shadow Cell Store	PostgreSQL 16	ShadowCell records: source grid, coord, visible_attrs, sync state, last_synced.	Indexed by (host_grid, source_grid)	Read on write on
Full-Text Search	Meilisearch / OpenSearch	Full-text index over row names, descriptions, tags, attribute text values, namespace paths.	Incremental index update on mutation	Search search
Time-Series Analytics	ClickHouse	High-throughput append of cell events, engagement metrics, mutation velocity. TimeAxis native support.	Column-oriented; partitioned by date	Append reads; n

Store	Technology	Responsibility	Encoding	Access
Snapshot Store	S3-compatible	Full Grid snapshots + per-cube checkpoints. Compressed JSON-LD. Signed for integrity.	Object store; versioned; lifecycle policy	Write: s manual;
AI Feature Store	Redis + PG materialized views	Pre-computed AI model inputs and outputs. Refreshed on schedule and on trigger.	Materialized views rebuilt per model TTL	Read: o Write: A

13.2 N-Dimensional Index Strategy

Index Type	Axes Covered	Query Types Accelerated	Storage Overhead
Composite B-Tree ($D_1 \times D_2$)	Dense cubes: $D_1 + D_2$ keys	Row lookup, column scan, 2D range scan	Low — standard PG index
N-dim Composite ($D_1 \dots D_N$)	All N axes	Full N-dim point lookup, partial prefix scan	$O(N \times \text{row_count})$ — use only for $N \leq 5$
JSONB GIN (attributes)	Cell attribute map	Attribute key existence, attribute value containment	Moderate — GIN index on JSONB
TimeSeries (BRIN on D_3)	TimeAxis keys (D_3)	Time range scans; chronological aggregation	Very low — BRIN is compact
Spatial (GeoHash on D_4)	GeoAxis keys (D_4)	Geographic bounding box, radius, nearest-neighbor	Low — PostGIS/GeoHash index
GIN on Tag/MultiEnum attrs	Tag, MultiEnum attribute keys	Attribute set membership, tag filtering	Moderate — GIN array index
Full-text (Meilisearch)	Text attribute keys (name, description)	Free-text search, phrase matching, faceted filters	Separate service; moderate storage
Graph adjacency (B-Tree)	HypergraphEdge from_node + to_node	$O(1)$ neighbor lookup, $O(\log n)$ edge traversal	Low — standard FK index
IdentityTag GIN	identity_tags array attribute	Partition-scoped queries, multi-identity filtering	Low-moderate
Namespace B-Tree	NamespacePath string column	Path prefix lookup, hierarchical namespace queries	Low — string prefix index
ZOrder curve (multi-dim)	$D_3 \times D_4$ (Time+Geo composite)	Range scans across two sparse axes simultaneously	Moderate — space-filling curve index

14. Security, Access Control & Compliance

14.1 N-Dimensional Security Model

Security in Hypergrid is enforced at five nested levels, from the Grid down to individual cell attributes:

Level	Enforcement Point	Mechanism	Bypass Protection
Grid	Authentication layer	JWT/API key validates tenant membership. Grid-level permissions granted.	No grid access without valid credential.
Space	SpaceMember permission tier	PermissionTier evaluated against SpaceMember.role on every request involving Space-scoped entities.	Role evaluated at API gateway. Cannot be elevated by application code.
Hypercube	CubeGovernance policy	CubeGovernance.access_policy evaluated before any cube-level operation (query, mutate, schema change).	PolicyEngine.evaluate() called before query planning.
HyperRow	Row-level security (PostgreSQL RLS + application)	VisibilityMask DimSlice predicate injected into all queries. HardIsolated partition rows have field-level encryption.	RLS enforced at PostgreSQL layer — no application-layer bypass.
Cell Attribute	Attribute-level visibility (application layer)	owner_only_attrs stripped from results for non-owner callers. Per-attribute visibility checked per-result-row.	Applied after RLS, before serialization. Cannot be bypassed by client.

14.2 Permission Tier Model

Tier	Ordinal	Capabilities
Public (unauthenticated)	0	Read cells where visibility=Public. No mutations. No graph traversal beyond public nodes.
Viewer	1	Read all permitted cells. No mutations. Can follow/subscribe (creates graph edge).
Subscriber	2	Read + bookmark + subscribe actions. No content mutations.
Contributor	3	Submit contributions to shared cubes. Comment. Vote. Cannot directly edit cells.
Editor	4	Edit cell values, add rows, add custom attribute keys to own cubes. Create DimSlice views.
Manager	5	Manage Space settings and members. Configure cube governance. Add new dimensions (requires governance gate for shared cubes).
Owner	6	Full ownership: all operations including dimension removal, cube deletion, identity configuration, federation setup.
Admin	7	Platform super-admin override. Legal hold access. Audit report generation.
System	8	Internal service-to-service. AI engine writeback. CRDT federation peer sync.

14.3 Data Governance for N-Dimensional Data

Governance Feature	Description	Enforcement Layer
Dimension-level Access Control	Individual axes can have axis_visibility settings. A Dim can be tenant-only (hidden from public queries) or identity-specific.	Query planner: invisible axes excluded from query scope before planning.
Attribute-level Encryption	owner_only_attrs are field-level AES-256 encrypted. Key derived from tenant identity. Never decrypted for non-owner queries.	PostgreSQL pgcrypto / application-level encryption before storage.
Time-axis Data Retention	TimeAxis cubes support configurable retention policies: delete cells older than N days/months.	Background job: purge cells outside retention window. EventLog preserves mutation history.
Schema Governance	Adding a new dimension or removing an attribute key requires a GovernanceProposal vote in multi-owner Spaces.	CubeGovernance.require_vote_for: [add_dimension, remove_attr, change_visibility]
Audit Trail	Every cell mutation, schema change, governance action, and identity operation appended to the immutable EventLog.	PostgreSQL append-only table. No UPDATE/DELETE permitted. RLS prevents tampering.
Right to Erasure (GDPR/CCPA)	Tenant can request erasure of personal data. Cell values anonymized; EventLog entries have actor_id replaced with anonymous_actor.	Erasure job: null personal attribute values + anonymize actor_id in EventLog. Cryptographic audit chain maintained.
Data Portability	Full Grid export in JSON-LD with schema definitions. Machine-readable, standards-aligned, signed for integrity.	HG-EXPORT: /export/grid/{grid_id}?format=jsonld — includes all cubes, axes, attrs, cells, graph, namespaces.
Cross-jurisdiction Data Residency	Per-tenant storage location configuration. Grid data confined to configured regions (AWS/GCP/Azure region selection).	Grid-level storage_config.region enforcement. Federation restricted to same-region peers by default.

15. API Surface (HG-API)

15.1 REST API — Core Endpoints

Endpoint	Method	Description	Min Auth
/grids	GET	List all Grids accessible to the caller	Viewer
/grids	POST	Create a new Grid	Admin
/grids/{id}/cubes	GET	List all Hypercubes in a Grid	Viewer
/cubes	POST	Create a new Hypercube	Editor
/cubes/{id}	GET	Get Hypercube schema and metadata	Viewer
/cubes/{id}/dims	GET	List all DimensionAxes for a Hypercube	Viewer
/cubes/{id}/dims	POST	Add a new DimensionAxis (N+1)	Manager
/cubes/{id}/attrs	GET	List AttributeKeyRegistry for a Hypercube	Viewer
/cubes/{id}/attrs	POST	Register a new attribute key	Editor
/cubes/{id}/rows	GET	Query HyperRows (DimSlice params + pagination)	Viewer
/cubes/{id}/rows	POST	Create a new HyperRow (set D ₁ key + initial attrs)	Editor
/cubes/{id}/rows/{d1_key}	GET	Get all cells for one HyperRow	Viewer
/cubes/{id}/rows/{d1_key}	PATCH	Update cell attribute values for a HyperRow	Editor
/cubes/{id}/rows/{d1_key}	DELETE	Soft-archive a HyperRow	Owner
/cubes/{id}/cells/{coord}	GET	Get a single HyperCell by N-dim coordinate	Viewer
/cubes/{id}/cells/{coord}	PUT	Set a HyperCell attribute map at a coordinate	Editor
/cubes/{id}/cells/{coord}/{attr_key}	PATCH	Update a single attribute of a HyperCell	Editor
/cubes/{id}/query	POST	Execute a HyperQL query	Viewer
/cubes/{id}/views	GET	List HypercubeViews	Viewer
/cubes/{id}/views	POST	Create a HypercubeView	Editor
/cubes/{id}/views/{view_id}/rows	GET	Render rows for a specific view (with all view config)	Viewer
/cubes/{id}/fold	POST	Execute a DimFold aggregation	Viewer

Endpoint	Method	Description	Min Auth
/cubes/{id}/expand	POST	Execute a DimExpand (pivot)	Viewer
/cubes/{id}/export	GET	Export cube (CSV/XLSX/JSON/Parquet/Arrow/JSON-LD)	Editor
/cubes/{id}/snapshot	POST	Create a cube snapshot	Owner
/cubes/{id}/restore/{snapshot_id}	POST	Restore cube from snapshot	Owner
/cubes/{id}/eventlog	GET	Get EventLog entries for this cube (paginated)	Editor
/cubes/{id}/crdt/sync	POST	Submit CrdtLog batch (federation peers only)	System

15.2 Graph, Space & Namespace API

Endpoint	Method	Description
/graph/nodes/{id}/tree	GET	Get link tree from node (depth, edge_type, direction params)
/graph/nodes/{id}/forest	GET	Get full link forest for all nodes owned by this identity
/graph/edges	POST	Create HypergraphEdge (triggers consent flow if required)
/graph/edges/{id}	DELETE	Delete edge (triggers shadow cell disconnection)
/graph/shadow-cells	GET	List all shadow cells in this grid from linked external grids
/graph/shadow-cells/{id}/sync	POST	Manually trigger shadow cell attribute sync from source
/graph/path	GET	Shortest path between two nodes (Dijkstra)
/graph/clusters	GET	Community cluster detection (Louvain)
/spaces	GET	List Spaces in this Grid
/spaces	POST	Create a Space
/spaces/{id}/members	GET	List Space members + roles
/spaces/{id}/governance/propose	POST	Submit governance proposal
/spaces/{id}/governance/vote	POST	Cast vote on proposal
/workspaces	POST	Create a Workspace

Endpoint	Method	Description
/workspaces/{id}/session	POST	Open Workspace session (returns WebSocket URL)
/workspaces/{id}/dim-context	PUT	Set workspace-level DimKey defaults (dim context)
/ns/resolve	GET	Resolve a NamespacePath to an entity ID
/ns	POST	Register a namespace entry
/ns/{id}/alias	POST	Add a namespace alias
/ns/{id}/federate	POST	Federate namespace entry to peer grid
/federation/peers	GET	List federation peers for this Grid
/federation/peers	POST	Register a new federation peer
/federation/sync	POST	Trigger manual federation sync with all peers
/identity/partitions	GET	List all TenantPartitions for authenticated identity
/identity/partitions	POST	Create a new TenantPartition
/identity/merge	POST	Initiate CrossTenantMerge (requires checkpoint + confirmation)
/ai/compute	POST	Request AI-computed attribute values for a batch of cells
/ai/nl-query	POST	Translate natural language to HyperQL
/ai/anomalies	GET	List AI-detected anomalies in a cube or DimSlice

15.3 WebSocket & Streaming

Channel	Description	Events
wss://.../cubes/{id}/stream	Real-time cell mutation stream for a Hypercube. Clients subscribe and receive all cell changes.	CellUpdated · RowCreated · RowArchived · DimAdded · AttrRegistered · AIWriteback · ConflictDetected
wss://.../workspaces/{id}/session	Workspace collaboration session. Cursor presence, cell lock indicators, co-editing synchronization.	CursorMoved · CellLocked · CellUnlocked · PresenceJoined · PresenceLeft · DimContextChanged
wss://.../spaces/{id}/activity	Space activity stream: member events, governance actions, graph edge changes.	MemberJoined · ProposalCreated · VoteSubmitted · EdgeCreated · ShadowCellUpdated

Channel	Description	Events
wss://.../graph/shadow-sync	Shadow cell synchronization stream. Receives mirror attribute deltas from linked external grids.	ShadowAttrUpdated · ShadowDisconnected · ShadowReconnected
wss://.../crdt/federation/{peer_id}	CRDT federation sync stream. Gossip CrdtOperations between peers.	CrdtOpBatch · VectorClockExchange · ConflictSurfaced · SchemaDelta

16. Plugin & Extension System (HG-PLUGIN)

HG-PLUGIN is the formal extension interface for Hypergrid. All platform-specific customizations — domain attribute sets, custom axis types, AI engine adapters, render mode adapters, data connectors, governance modules, and export adapters — are registered as HypercubePlugin implementations. Plugins are loaded at Grid initialization and can be updated without downtime.

16.1 Plugin Interface

```
// The HypercubePlugin trait — all Hypergrid extensions implement this
pub trait HypercubePlugin: Send + Sync {
    fn plugin_id(&self)    -> PluginId;
    fn plugin_name(&self) -> &str;
    fn version(&self)      -> semver::Version;
    fn capabilities(&self) -> Vec<PluginCapability>;

    // Lifecycle
    fn on_load(&mut self, ctx: &PluginContext) -> Result<(), PluginError>;
    fn on_unload(&mut self)                    -> Result<(), PluginError>;
    fn health_check(&self)                     -> PluginHealth;
}

// Plugin capability mix-ins — implement the ones your plugin provides
pub trait AxisTypePlugin: HypercubePlugin {
    fn axis_type_id(&self) -> AxisType;
    fn validate_key(&self, key: &DimKey) -> Result<(), ValidationError>;
    fn compare_keys(&self, a: &DimKey, b: &DimKey) -> Ordering;
    fn key_to_display(&self, key: &DimKey) -> String;
    fn index_strategy(&self) -> IndexStrategy;
}

pub trait AttributeTypePlugin: HypercubePlugin {
    fn attr_type_id(&self) -> AttributeType;
    fn validate_value(&self, v: &TypedAttrValue) -> Result<(), ValidationError>;
    fn merge(&self, a: &TypedAttrValue, b: &TypedAttrValue,
              ctx: &MergeContext) -> TypedAttrValue;
    fn render_cell(&self, v: &TypedAttrValue) -> RenderCell;
}

pub trait ComputedModelPlugin: HypercubePlugin {
    fn model_id(&self) -> ModelId;
    fn output_attrs(&self) -> Vec<AttributeKeyDef>;
    fn compute(&self, row: &HyperRow, ctx: &ComputeContext) -> ComputeResult;
    fn invalidation_triggers(&self) -> Vec<EventKind>;
    fn cache_ttl(&self) -> Duration;
}
```

```

pub trait RenderModePlugin: HypercubePlugin {
    fn render_mode_id(&self) -> RenderMode;
    fn render(&self, view: &HypercubeView, data: &ProjectedCube) -> RenderOutput;
    fn supported_axes(&self) -> Vec<AxisType>;
}

pub trait DataConnectorPlugin: HypercubePlugin {
    fn connector_type(&self) -> ConnectorType;
    fn import(&self, source: ConnectorSource) -> Result<ImportPlan,
ConnectorError>;
    fn export(&self, cube: &HypercubeSlice, target: ConnectorTarget) -> Result<(),
ConnectorError>;
    fn sync_stream(&self) -> Option<Box<dyn SyncStream>>;
}

pub trait AIEnginePlugin: HypercubePlugin {
    fn ai_engine_id(&self) -> AIEngineId;
    fn register_model(&self, model: AIModelDef) -> ModelId;
    fn compute_batch(&self, reqs: Vec<ComputeRequest>) -> Vec<ComputeResult>;
    fn detect_anomalies(&self, slice: &DimSlice, attr: &AttributeKey) ->
Vec<AnomalySignal>;
    fn nl_to_query(&self, nl: &str, ctx: &CubeContext) -> HyperQLQuery;
}

pub trait GovernancePlugin: HypercubePlugin {
    fn governance_model_id(&self) -> GovernanceModelId;
    fn evaluate_proposal(&self, proposal: &Proposal) -> GovernanceDecision;
    fn quorum_reached(&self, votes: &[Vote], total_members: u32) -> bool;
    fn compute_distribution(&self, contributors: &[Contributor]) ->
Vec<Distribution>;
}

```

16.2 Reference Plugin Catalog

Plugin Name	Type	Description
time-axis	AxisType	Built-in TimeAxis plugin: Timestamp keys, BRIN index, chronological ordering, rollup by period, AS_OF time-travel support
geo-axis	AxisType	Built-in GeoAxis plugin: GeoHash keys, PostGIS index, bounding box and radius queries, NUTS/ISO admin level hierarchies
hierarchy-axis	AxisType	Built-in HierarchyAxis plugin: HierarchyPath keys, parent-child validation, ROLLUP and DRILLDOWN computation, closure table index
version-axis	AxisType	Built-in VersionAxis plugin: SemVer keys, fork/merge operations, diff computation between version keys

Plugin Name	Type	Description
openai-ai-engine	AIEngine	Reference AI engine adapter: OpenAI-compatible API. NL-to-HyperQL, anomaly detection, health scoring, attribute completion
pm-domain	ComputedModel	Project management attribute pack: health_score, schedule_risk, budget_risk, predicted_completion (ComputedModelPlugin)
finance-domain	ComputedModel	Financial domain attribute pack: profit, roi, ytd_actuals, budget_variance, cash_flow_forecast
crm-domain	ComputedModel	CRM attribute pack: lead_score, pipeline_value, churn_probability, engagement_health
kanban-renderer	RenderMode	Kanban board renderer: swimlane columns = enum axis values; cards = row entities; WIP limits; drag-and-drop events
gantt-renderer	RenderMode	Gantt chart renderer: horizontal bars on TimeAxis; Dependency edges as arrows; critical path; milestones
ndim-explorer	RenderMode	N-Dim interactive explorer: axis selection dropdowns; DimSlice sliders; real-time DimFold/DimExpand
network-graph-renderer	RenderMode	Force-directed graph renderer: HG-GRAPH edges as visual links; D3.js + WebGL for large graphs
csv-connector	DataConnector	CSV import/export: column-to-attr mapping wizard, type inference, batch row creation, streaming export
excel-connector	DataConnector	XLSX import/export: multi-sheet mapping, formula preservation, chart data export
parquet-connector	DataConnector	Apache Parquet export: columnar format, Snappy compression, schema embedding — for data lake pipelines
graphql-connector	DataConnector	GraphQL API adapter: expose any Hypercube as a GraphQL schema; mutation subscriptions
cooperative-governance	Governance	Cooperative democracy governance: 1-member-1-vote, Shapley value distribution, multi-sig treasury
democracy-governance	Governance	Democratic governance: configurable quorum, supermajority options, ranked-choice voting
multisig-treasury	Governance	Multi-signature treasury: M-of-N approval for financial operations; escrow support

17. Export & Integration (HG-EXPORT)

17.1 Export Formats

Format	Content	N-Dim Handling	API Endpoint
CSV	Flat 2D view of any HypercubeView (DimFold applied to reduce to 2D). One row per D_1 entity, one column per D_2 attr (+ folded dim values as additional columns).	DimFolds reduce $N \rightarrow 2$ before export. DimExpands create extra columns.	<code>/cubes/{id}/export?format=csv&view_id=..</code>
XLSX	Multi-sheet Excel workbook: one sheet per unique combination of $D_3 \dots D_N$ keys (if $N \leq 4$); or one sheet per DimFold configuration.	Multiple sheets for different dim slices. Pivot table per DimExpand.	<code>/cubes/{id}/export?format=xlsx</code>
JSON	Full N-dim JSON: cube schema + array of HyperCell objects with full DimCoordinate and AttributeMap.	Full fidelity: all N dimension keys and all attributes per cell.	<code>/cubes/{id}/export?format=json</code>
JSON-LD	Linked Data export with schema.org / custom vocab type annotations. Machine-readable and semantically rich.	DimAxis types mapped to schema.org dimensions. Full coordinates.	<code>/cubes/{id}/export?format=jsonld</code>
Parquet	Apache Parquet columnar format: each attribute key is a column; each dimension key is a partition column.	Natural fit: Parquet supports partition columns for each dim axis.	<code>/cubes/{id}/export?format=parquet</code>
Arrow	Apache Arrow in-memory columnar format for high-performance data pipeline integration.	Dimension keys as partition columns; attribute values as record columns.	<code>/cubes/{id}/export?format=arrow</code>
GraphQL	GraphQL schema + resolvers auto-generated from cube schema. Mutations and subscriptions included.	DimSlice args on every GraphQL query. N-dim pagination built-in.	<code>/cubes/{id}/graphql</code> — auto-generated schema
Webhook	Real-time push of cell mutation events to a registered external URL on every EventLog append.	Events include full DimCoordinate and changed AttributeMap.	<code>/cubes/{id}/webhooks</code> — manage subscriptions
Embed	Embeddable iframe/script: read-only or edit-enabled view of any HypercubeView in an external webpage.	Respects all DimSlice + VisibilityMask settings of the view.	<code>/cubes/{id}/views/{view_id}/embed</code>
GDPR Archive	Complete Grid export: all cubes, all axes, all cells, EventLog, graph, namespaces. Encrypted, signed.	Full N-dim fidelity. Compressed. Verifiable chain of custody.	<code>/grids/{id}/export/gdpr-archive</code>

17.2 Import Sources

Source	Method	N-Dim Mapping	Dim Assignment
CSV / XLSX	File upload → AI-assisted attribute mapping wizard → batch HyperRow creation	Single dim by default ($D_1 \times D_2$). Multi-sheet XLSX: each sheet = one D_3 slice.	Wizard prompts for D_3 – D_n key assignment (default or auto-detect)
Relational DB	JDBC/ODBC connector → table-to-cube mapping → schema inference	Table cols → D_2 attrs. FK relationships → HG-GRAPH edges. Views → DimSlice configs.	D_3 – D_n from table partitions or timestamp columns
REST / GraphQL API	API connector → response mapping → incremental sync (webhook or polling)	JSON response fields → D_2 attrs. Nested objects → custom D_3 expansion.	API response pagination key D_3 if time or ordinal
Apache Kafka	Kafka consumer → streaming cell mutations. Real-time event sourcing into cube.	Event fields → attrs. Kafka topic partition → custom axis key.	Kafka offset → D_3 (sequence axis) or timestamp → D_3 (time axis)
Parquet / Arrow	Object store pull → columnar load → high-throughput batch insert	Parquet partition columns → dim axis keys. Record columns → attrs.	Auto-inferred from Parquet schema
HyperQL Feed	Another Hypergrid instance pushes via CrossGridLink + federation sync protocol.	Full N-dim fidelity: DimCoordinates preserved. Attrs synchronized.	Source dim keys preserved; local axis aliases configurable
Git Repository	Git connector: commits → rows, changed files → cells, branches → D_3 (version axis).	D_1 =commit, D_2 =file metric, D_3 =branch/tag	Branch as version axis; commit timestamp as time sub-axis

18. Performance, Scalability & Operations

18.1 Performance Targets

Operation	N	Target p50	Target p99	Key Optimization
2D cell query (1K rows)	2	<30ms	<150ms	Dense encoding; hot CellCache; result compression
N-dim query with 2 DimSlices	4	<80ms	<400ms	Composite N-dim index; DimSlice predicate pushdown to PG
DimFold + DimExpand (pivot)	4	<200ms	<1s	Materialized DimFold views for common pivot configurations
HyperQL complex join	3	<300ms	<2s	Query planner cost estimation; index-only scans; join reordering
CRDT merge (1K ops)	-	<80ms	<400ms	Rust native CRDT; batched merge for federation peers
Shadow cell sync (single attr)	-	<300ms	<2s	Kafka priority queue; WebSocket push for active sessions
Namespace resolution	-	<5ms	<20ms	Redis cache (300s TTL); pre-warm on Space load
Graph tree traversal (depth 3)	-	<150ms	<800ms	Closure table for HierarchyAxis; adjacency list for others
AI attribute compute (batch 100)	-	<500ms	<5s	AI engine batching; CellCache pre-populate; circuit breaker
Full cube export (100K rows, N=3)	3	<10s	<60s	Streaming export; server-side DimFold before serialization

18.2 Horizontal Scalability Architecture

Component	Scale Strategy	Limits	Scale Trigger
Cell Store (PostgreSQL)	HASH partitioning by (grid_id, cube_id). Read replicas for queries. PgBouncer connection pool.	10B cells per deployment (estimated)	CPU > 70% on primary; replica lag > 1s
CRDT Service	Horizontal scale: stateless (VectorClock in Redis). Kafka for async persistence. Multiple instances.	500K CRDT ops/sec per cluster	Queue depth > 10K; latency p99 > 200ms
Graph Store	PostgreSQL for hot graphs (< 10M edges). Neo4j/DGGraph for full global graph at scale.	1B edges (Neo4j); 100M edges (PostgreSQL)	Graph traversal p99 > node count > 10M
AI Compute Tier	AI engine pods auto-scale on compute queue depth. Circuit breaker prevents cascade on model unavailability.	Model-dependent (API rate limits apply)	Compute queue > 1K requests; p99 > 2s

Component	Scale Strategy	Limits	Scale Trigger
WebSocket (Workspace)	Sticky sessions on workspace_id. Redis pub/sub for fanout. Horizontal scale with consistent hashing.	100K concurrent sessions per cluster	>80% WS connection capacity
Namespace Cache (Redis)	Redis Cluster: 6 shards. LRU eviction. Pre-warm on Space load. CDN-cached for public paths.	1B cached paths (estimated)	Cache miss rate > 20% eviction rate spike
Kafka (Event streaming)	Topic partitioned by (grid_id % num_partitions). Consumer group per service. Dead letter queue per group.	10M events/sec per cluster	Consumer lag > 5K; to partition imbalance
Federation Sync	Per-peer sync worker. Isolated queue per federation peer. Back-pressure on slow peers.	255 federation peers per Grid	Peer delta > 500K ops sync lag > 5 min

18.3 Observability

Signal	Tool	Key Metrics	Alerts
Metrics	Prometheus + Grafana	Query latency by (cube_id, N, operation_type); CRDT merge rate; cell cache hit/miss; AI compute queue depth; graph traversal latency; shadow sync lag	p99 > threshold; cache miss > 20%; queue depth spike
Traces	OpenTelemetry + Jaeger	Per-request distributed trace: grid_id, cube_id, N, dim_slices, identity_tag, space_id, AI model calls	Trace > 2s; AI model call fails
Logs	Structured JSON → Loki	All HyperQL executions, cell mutations, CRDT ops, governance actions, identity events, graph edge changes	Error rate spike; auth failure burst
Audit	Immutable EventLog	Every cell mutation with (actor, coord, attr, old_value, new_value, vector_clock, identity_tag)	Cross-partition access; owner permission change; dimension removal
Synthetic	k6 / playwright	E2E cube query test (every 5 min); CRDT sync test; shadow cell sync test; namespace resolution test	Any synthetic > SLA; federation sync failure

19. Platform Integration Guide

Hypergrid is designed to be embedded in any platform as the universal data substrate. This section provides a practical guide for platform teams adopting Hypergrid, covering initialization, schema design, the N-dimensional modeling methodology, and common integration patterns.

19.1 Adoption Checklist

Step	Description	Responsible	Outcome
1. Grid Provisioning	Create a Grid for your platform. Configure: grid_name, default tenant model, storage region, AI engine adapter, plugin set.	Platform architect	Grid running; system cubes provisioned
2. Domain Modeling	Identify your domain entities ($\rightarrow D_1$ EntityAxis), properties ($\rightarrow D_2$ PropertyAxis), and additional contexts ($\rightarrow D_3$ — D_n custom axes).	Domain architect	N-dim schema design document
3. Attribute Pack Design	Define your domain AttributeKeyRegistry. Specify types, CRDT semantics, visibility, aggregation, and computation for each key.	Domain architect	AttributeKeyRegistry registered
4. Plugin Registration	Register domain plugins: computed model plugins, custom axis type plugins (if any), AI model definitions.	Platform engineer	All plugins loaded and healthy
5. Space Types	Configure your Space type taxonomy. Define default cube provisioning per Space type. Set governance defaults.	Platform architect	SpaceType registry configured
6. Template Cubes	Define template Hypercubes (pre-configured axis + attribute schema) for common use cases. Publish to template library.	Platform team	Template library available to tenants
7. Permission Model	Map your platform's permission model to Hypergrid PermissionTiers. Configure VisibilityMask defaults per Space type.	Security architect	Permission model documented + tested
8. SDK Integration	Integrate the HG-SDK into your platform's backend and frontend. Configure API gateway routing to Hypergrid services.	Platform engineer	SDK integrated; API routes configured
9. Migration	Import existing data: use HG-IMPORT connectors or HyperQL INSERT for initial data load. Set up incremental sync if needed.	Data engineer	Historical data imported + validated
10. Federation Setup	If multi-tenant or multi-region: configure federation peers. Set CRDT sync intervals. Test delta sync with test data.	Platform engineer	Federation sync healthy; conflict rate < 0.1%

19.2 N-Dimensional Modeling Methodology

Follow this decision framework when designing the axis structure for a new Hypercube:

Question	Answer → Design Decision
What is the primary entity being tracked?	→ D ₁ EntityAxis. Each row = one entity. Choose key type (UUID for new entities; existing ID for migration).
What properties does each entity have?	→ D ₂ PropertyAxis. Each column = one property. Register as attribute keys in AttributeKeyRegistry.
Do I need to track how properties change over time?	→ Yes: add D ₃ TimeAxis. Each cell now holds a (entity, property, time_period) data point.
Do I need a geographic or regional breakdown?	→ Yes: add D ₄ GeoAxis. Use GeoHash or NUTS/ISO admin level keys.
Do I need to compare scenarios, plans, or versions?	→ Yes: add D ₅ ScenarioAxis or VersionAxis. Keys: "base_case", "optimistic", "forecast", "actuals", etc.
Do I need org/department breakdowns?	→ Yes: add D ₆ HierarchyAxis (OrgUnit). Parent-child relationships defined in hierarchy. ROLLUP enabled.
Am I adding a dimension just to filter (not pivot/aggregate)?	→ No: use a D ₂ attribute key (filter column) instead of a new axis. Keep N small.
Will most (D ₁ , D ₂ , D ₃ , ...) combinations have a value?	→ Yes: use Dense encoding. No (many null combinations): use Sparse encoding.
Is N > 6 in my design?	→ Warning: reconsider. Flatten some axes into attributes. Complex N-dim cubes are hard to query and visualize.
Do I need cells to be addressable by namespace?	→ Yes: register HyperRows in HG-NS. NamespacePath: hypergrid://{grid}/{cube}/row/{d1_key}/

19.3 Common Integration Patterns

Pattern	N	Description	Example Domain
Entity ledger	2	D ₁ =entity, D ₂ =property. Simple key-value record store with governance and CRDT.	CRM contacts, Product catalog, User profiles
Timeseries cube	3	D ₁ =entity, D ₂ =metric, D ₃ =timestamp. Track how entity metrics change over time.	Financial time-series, IoT sensor readings, Analytics metrics
Geo cube	3	D ₁ =entity, D ₂ =metric, D ₃ /D ₄ =geo. Regional performance breakdown.	Sales by region, Public health by geography, Weather data
Planning matrix	4	D ₁ =entity, D ₂ =metric, D ₃ =time, D ₄ =scenario. Track actuals vs plan vs forecast.	FP&A planning, Budget vs actuals, Capacity planning
Org hierarchy cube	4	D ₁ =employee, D ₂ =attribute, D ₃ =time, D ₄ =org_unit (hierarchy). Roll up people metrics across org structure.	HR analytics, Compensation planning, Performance management

Pattern	N	Description	Example Domain
Dependency graph cube	3	D ₁ =component, D ₂ =attribute, D ₃ =version. Plus HG-GRAPH Dependency edges for critical path.	Software dependency management, Supply chain, Project dependencies
Research data cube	5	D ₁ =sample, D ₂ =measurement, D ₃ =time, D ₄ =condition, D ₅ =researcher. Multi-condition experimental data with full provenance.	Clinical trials, Lab experiments, Market research
Marketplace exchange	4	D ₁ =listing, D ₂ =attribute, D ₃ =time, D ₄ =market_segment. Track listing performance across time and market segment.	E-commerce, Financial exchange, Labor marketplace
Federated workspace	2	N=2 cube but federated across multiple Grid deployments via CrossGridLink + ShadowCell protocol.	Inter-company collaboration, Open-source project tracking, Partner data sharing
Multi-tenant SaaS	3	D ₁ =entity, D ₂ =attribute, D ₃ =tenant_id (TenantAxis). Single physical cube serving multiple tenants with RLS per D ₃ key.	SaaS platform shared infrastructure, White-label products, Multi-org portals

20. Open Items, Decisions & Roadmap

20.1 Open Technical Decisions

Decision	Options	Recommendation	Decision By
N-dim index at scale ($N \geq 4$)	PostgreSQL composite index Z-order curve Specialized OLAP DB (DuckDB, ClickHouse) Hybrid	Hybrid: PG for $N \leq 4$; ClickHouse for $N > 4$ analytics cubes	v1.1 milestone
CRDT conflict surfacing UX	Silent (last-write-wins always) Notify (user review on conflict) Block (require resolution)	Notify for semantic conflicts (status lattice); Silent for counters/sets	v1.0 milestone
Shadow cell write-back governance	Open (any editor) Governed (proposal required) Opt-in per attribute	Opt-in per attribute: write_back_attrs configurable per CrossGridLink	v1.0 milestone
Max N (dimensionality limit)	8 12 16 Unlimited	Hard limit 16; recommended max 6 for UI usability	v1.0 (finalize)
Formula engine sandboxing	WASM sandbox Process isolation Interpreted AST JVM sandbox	Interpreted AST with resource limits (no arbitrary code)	v1.0 milestone
Federation identity model	Grid-level trust Namespace-level trust Identity-level trust All three tiers	All three: Grid(full) > Namespace(partial) > Identity(minimal)	v1.5 milestone
AI engine default	OpenAI-compatible API Local LLM (Ollama) No default (platform-provided only) Multiple	Multiple: OpenAI-compatible + local LLM option + no-AI mode	v1.0 milestone
Time-travel granularity	Day Hour Minute Second Millisecond EventLog entry (highest fidelity)	EventLog entry: AS_OF any timestamp resolves to last EventLog entry before that time	v1.0 milestone

20.2 Version Roadmap

Version	Milestone	Key Deliverables
v0.5 (Alpha)	Core substrate	HyperCell model · $N=2$ dense encoding · AttributeKeyRegistry · EventLog · PostgreSQL persistence · Basic REST API · LWW CRDT
v1.0 (Beta)	2D+ platform	$N=1-6$ dimensions · Sparse + Hybrid encoding · HyperQL v1 · Formula language · View Engine (10 render modes) · Federation v1 · Spaces v1 · HG-NS v1 · Go SDK + TypeScript SDK
v1.1	Full N-dim + Graph	All 16 dimension types · HG-GRAPH full implementation · CrossGridLink + Shadow Cell protocol · Link forest traversal · N-dim index optimization · ClickHouse analytics integration
v1.5	Intelligence + Identity	HG-AI pluggable engine · OpenAI-compatible adapter · NL-to-HyperQL · Anomaly detection · HG-ID multi-tenant identity · TenantPartition · VisibilityMask · CrossTenantMerge

Version	Milestone	Key Deliverables
v2.0	Full platform	All render modes including N-DimExplorer · Governance plugins (cooperative, democracy, multisig) · All export formats (Parquet, Arrow, GraphQL) · Plugin marketplace · Web-based N-dim exploration UI
v2.5	Scale + Federation	Neo4j graph backend for scale · Cross-Grid AI (federated ML models) · ZOrder curve multi-dim index · 255-node federation · GDPR/CCPA automated compliance workflows
v3.0	Autonomous Intelligence	Autonomous AI agents operating on cubes with approval-first workflows · Predictive DimFold models · Cross-Grid MatchEngine · Semantic layer (AI-generated cube descriptions + ontology mapping) · On-chain identity anchoring (DID)

20.3 Glossary

Term	Definition
Hypergrid	N-Dimensional Distributed Spreadsheet System — the complete platform described in this document
Grid	Root container for all Hypercubes, dimensions, tenants, namespaces, spaces, and graph structures
Hypercube	An N-dimensional grid $H=(D_1, D_2, \dots, D_N)$. The generalization of a spreadsheet sheet.
DimensionAxis (D_i)	One axis of a Hypercube: EntityAxis (D_1), PropertyAxis (D_2), or any custom type (D_3-D_N)
DimCoordinate	An ordered tuple of N dimension keys, one per axis, uniquely addressing a HyperCell
HyperCell	A data point at an N-dimensional coordinate, carrying an N-attribute map
HyperRow	All cells sharing the same D_1 key — the canonical entity in a Hypercube
AttributeMap	A <code>HashMap<AttributeKey, TypedAttrValue></code> — the N-attribute payload of a HyperCell
AttributeKey	A registered string key in the AttributeKeyRegistry. The "column name" generalized to apply across all dimensions.
DimSlice	A predicate applied to one or more dimension axes, producing a sub-cube or projection
DimFold	Collapsing an axis by aggregating all its key values into a summary. Generalization of GROUP BY.
DimExpand	Expanding a dimension's key set as separate columns. Generalization of PIVOT.
HypercubeView	A saved, shareable configuration of DimSlices, DimFolds, axis remappings, filters, sorts, and render mode
Hypergraph	The graph layer: typed, directed edges between HyperCells, HyperRows, Cubes, Spaces, and Grids
CrossGridLink	A HypergraphEdge crossing Grid boundaries. The inter-grid link network atom.

Term	Definition
ShadowCell	A read-only reflection in Grid A of a linked entity from Grid B, created by a CrossGridLink
MirrorAttribute	A specific attribute from a ShadowCell synced into the host Grid as a read-only computed attribute
LinkForest	The complete set of all link trees rooted at a given identity — their full cross-grid network
Space	A named, governed, bounded operational context within a Grid. Groups cubes, members, and governance.
Workspace	Active working session within a Space. Personalized cube/view arrangement + session state.
NamespacePath	Hierarchical URI addressing any entity: hypergrid://{grid}/{space_type}/{slug}/...
SovereignTenant	The real-world entity (person, org, or agent) that owns a root set of Hypercubes and Spaces
TenantPartition	A logical partition of a SovereignTenant's data, scoped to one Identity. Row-level via identity_tags.
VisibilityMask	N-dimensional DimSlice predicates defining what is visible to each observer type
VectorClock	HashMap<NodeId, u64>. Logical clock for causal ordering of all mutations across federation nodes
CRDT	Conflict-free Replicated Data Type — per-attribute semantics: LWW OR-Set Counter Lattice
HyperQL	The N-dimensional query language of Hypergrid: SELECT, DimSlice WHERE, FOLD, EXPAND, TRAVERSE GRAPH, AS_OF
HypercubePlugin	The extension interface for all Hypergrid customizations: axis types, attr types, AI engines, render modes, connectors
AIEngineAdapter	Plugin interface for connecting any AI/ML service to provide Tier-2 computed attributes
ComputedAttribute	A derived attribute value: Tier-1 (synchronous formula) or Tier-2 (asynchronous AI/ML signal)
WritebackProtocol	The secure channel by which AI engines write computed attribute values into HyperCells, respecting all permission and audit requirements
AS_OF	Time-travel query operator: AS_OF timestamp returns the state of the cube at that historical moment by replaying the EventLog
EventLog	Append-only, immutable log of every mutation. Every cell attribute change is an EventLog entry.
CrdtLog	In-flight CRDT operation buffer: operations pending application and federation peer sync
Federation	CRDT synchronization between two or more Grid deployments. Delta sync via Kafka; VectorClock-based causal ordering
N-DimExplorer	Interactive browser for N-dimensional data: axis selection, DimSlice sliders, live DimFold/DimExpand, graph overlay